

基于语言机制的内核模块隔离研究

姓名： 陈林峰

学号： 3220231213

学院： 计算机学院

2026 年 6 月

①

基于语言机制的内核模块隔离研究

陈林峰

北京理工大学

中图分类号: TQ028.1

UDC 分类号: 540

学校代码: 10007

★ 学生类型

☐ 工程硕博士专项

☐ 交叉研究方向

☐ 政府项目留学生

基于语言机制的内核模块隔离研究

作 者 姓 名	陈林峰
二 级 培 养 单 位	计算机学院
指 导 教 师	陆慧梅教授
答辩委员会主席	李凡教授
申 请 类 别	专业学位硕士
学 位 领 域	电子信息
学位授予单位	北京理工大学
论文提交日期	2026 年 6 月

Research on Kernel Module Isolation Based on Language Mechanism

Candidate Name:	<u>Chen Linfeng</u>
School or Department:	<u>Beijing Institute of Technology</u>
Faculty Mentor:	<u>Prof. Huimei Lu</u>
Chair, Thesis Committee:	<u>Prof. Fan Li</u>
Degree Applied:	<u>Professional Degree of Master</u>
Major:	<u>Electronic and Information</u>
Degree by:	<u>Beijing Institute of Technology</u>
Submission date of the paper:	<u>June, 2026</u>

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下独立完成的科研成果。文中所撰写内容符合以下学术规范（请勾选）：

☒ 论文综述遵循“适当引用”的规范，全部引用的内容不超过 50%。

☒ 论文中的研究数据及结果不存在篡改、剽窃、抄袭、伪造等学术不端行为，并愿意承担因学术不端行为所带来的一切后果和法律责任。

☒ 文中依法引用他人的成果，均已做出明确标注或得到许可。

☒ 论文内容未包含法律意义上已属于他人的任何形式的研究成果，也不包含本人已用于其他学位申请的论文或成果。

☒ 与本人一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此声明。

签 名：

日 期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：

① 学校有权保管、并向有关部门送交学位论文的原件与复印件；

② 学校可以采用影印、缩印或其它复制手段复制并保存学位论文；

③ 学校可允许学位论文被查阅或借阅；

④ 学校可以学术交流为目的，复制赠送和交换学位论文；

⑤ 学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名：

日 期：

导师签名：

日 期：

致谢

岁月流转，行文至此，心中最先涌起的是感恩。本论文从选题、调研、设计到实现与撰写，凝结的不仅是个人的努力，更离不开许多师长、同学与亲友的支持与陪伴。

首先，衷心感谢我的导师。在论文研究与写作过程中，导师始终以严谨求实的学术态度和细致耐心的指导帮助我不断完善思路。无论是在研究方向的把握、技术路线的选择，还是在论文结构与表达的打磨上，导师都给予了我极大的启发与鼓励。导师对学术规范的坚守、对研究细节的重视以及对学生成长的关怀，使我受益良多，这份影响也将长久伴随我未来的学习与工作。

感谢课题组的各位老师与同学。他们在学术讨论中提出的宝贵建议，在实验推进中给予的支持，以及在遇到困难时的真诚帮助，都是我顺利完成本论文的重要力量。与他们并肩学习、共同进步的时光，是研究生阶段最珍贵的记忆之一。

感谢学院与学校提供的良好科研与学习环境，为本课题的开展创造了坚实条件；感谢在资料收集、实验测试与论文排版过程中提供帮助的老师与同学，他们的付出使本论文得以更加完整。

最后，我要特别感谢我的家人。感谢他们一直以来的理解、包容与支持，在我焦虑与迷茫时给予我温暖与力量，在我坚持不下去时鼓励我继续前行。他们无声而坚定的陪伴，是我完成学业最坚实的后盾。

谨以此文，向所有关心、帮助和支持过我的师长、同学、朋友与家人致以最诚挚的谢意。

基于语言机制的内核模块隔离研究

摘要：操作系统内核是计算机系统的核心，其安全性对系统的稳定运行至关重要。由于历史遗留的限制，Linux 等通用操作系统在架构设计中缺乏对安全性的考虑，许多内核模块因出现内存安全问题而导致系统崩溃，同时，复杂的模块间依赖关系使得模块的热更新难以实现。近年来，许多工作通过特定的硬件扩展或者基于软件方法来实施内核模块的隔离，从而缓解内存安全问题对内核的影响，然而，这些方案因性能开销或灵活性不足难以适用于内核模块。随着编程语言的演进，诸如 Rust 等兼具高性能和内存安全的现代语言为实现高效、轻量化且细粒度的隔离提供了新思路。

本文提出了 MOAT，一个通用、灵活、低开销的内核模块隔离框架，通过 Rust 语言提供的类型与内存安全，实现语言驱动的内核模块隔离与热升级，这些具有隔离属性的模块称为隔离域。MOAT 有几个关键的新颖之处，首先，基于系统分解和模块化思想，MOAT 将隔离所需的基础设施与隔离域的实现解耦，将系统划分为可信计算基（TCB）和域集合两部分，这使得 MOAT 可以轻松移植到不同的操作系统中。针对以往基于语言的隔离研究，Moat 进一步细化了安全代码和不安全代码的边界，并将 TCB 划分为域管理运行时和域基础库。通过在域基础库中对低级内核资源的所有不安全操作进行安全封装，所有的域集合可以完全使用安全的代码实现。其次，为了实现完整的内存隔离和故障隔离，Moat 设计了一套用于隔离域之间以及隔离域与原生内核模块之间通信的通信机制。最后，MOAT 利用状态数据的分离存储实现隔离域的热升级，基于读-复制-更新（Read-Copy-Update）和优化的读写锁机制，为有状态和无状态隔离域设计了热更新策略，以最大限度地缩短更新时的停机时间。

为了评估所提出的设计，本文构建了一个名为 AlienOS 的 Rust 内核，其由 29 个隔离域构成，并基于 MOAT 对 Linux 内核进行了安全重构。实验结果表明，MOAT 的跨域调用速度对比方案快三倍，而吞吐量开销仅为 1% 至 2%。此外，所提出的实时更新策略对比方案实现了更短的服务暂停时间。

关键词：操作系统；内核模块隔离；热升级；内存安全；编程语言

Research on Kernel Module Isolation Based on Language Mechanism

Abstract: The operating system kernel is the core of a computer system, and its security is critical to the stable operation of the entire system. Due to historical constraints, general-purpose operating systems such as Linux were not originally designed with strong security considerations. As a result, many kernel modules can crash the system when memory safety violations occur. Moreover, the complex dependency relationships among modules make live updates difficult to implement. In recent years, numerous studies have attempted to isolate kernel modules through hardware extensions or software-based techniques to mitigate the impact of memory safety vulnerabilities on the kernel. However, these approaches are often unsuitable for kernel modules due to either performance overhead or insufficient flexibility. With the evolution of programming languages, modern languages such as Rust, which combine high performance with memory safety, provide new opportunities for achieving efficient, lightweight, and fine-grained isolation.

This paper presents MOAT, a general-purpose, flexible, and low-overhead kernel module isolation framework. Leveraging Rust's type and memory safety guarantees, MOAT enables language-driven kernel module isolation and live updates. Modules with isolation properties are referred to as isolation domains. MOAT introduces several key innovations. First, following the principles of system decomposition and modularization, MOAT decouples the infrastructure required for isolation from the implementation of isolation domains. The system is divided into two parts: a Trusted Computing Base (TCB), also referred to as the isolation infrastructure, and a set of domains. This design allows MOAT to be easily ported to different operating systems. Building on prior research on language-based isolation, MOAT further refines the boundary between safe and unsafe code by partitioning the TCB into a domain management runtime and a domain base library. By providing safe abstractions for all low-level kernel resource operations within the domain base library, all domains can be implemented entirely in safe Rust. Second, to achieve comprehensive memory isolation and fault isolation, MOAT designs a communication mechanism that supports interactions both among isolation domains and between isolation domains and native kernel modules. Finally, MOAT

enables live updates of isolation domains through separated storage of state data. Based on Read-Copy-Update (RCU) and an optimized read-write lock mechanism, MOAT provides live update strategies for both stateful and stateless isolation domains, thereby minimizing service downtime during updates.

To evaluate the proposed design, we built a Rust-based kernel named AlienOS, consisting of 29 isolation domains, and also performed a secure refactoring of the Linux kernel based on MOAT. Experimental results show that MOAT achieves cross-domain invocation performance three times faster than the comparison approach, while incurring only 1%–2% throughput overhead. Furthermore, the proposed live update strategy achieves significantly shorter service pause times compared with existing approaches.

Key Words: Operating system; kernel module isolation; live updates; memory safety; programming languages

目次

1	绪论	1
1.1	研究的背景和意义	1
1.2	本文的研究内容	2
1.3	本文的组织结构	3
2	研究现状	5
2.1	基于硬件机制的隔离	5
2.2	基于软件方法的隔离	7
2.3	基于传统安全语言的隔离	8
2.4	基于 Rust 语言的隔离	9
2.5	热更新	12
2.6	总结和目标	14
2.7	本章小结	15
3	Moat 内核模块隔离框架设计	16
3.1	内核模块隔离的总体设计思路	16
3.2	隔离域集合	18
3.3	隔离域基础库	19
3.3.1	基础库的基本功能	19
3.3.2	基础库的安全抽象	21
3.4	隔离域管理运行时	23
3.5	域的隔离属性	25
3.5.1	域间通信与隔离	26
3.5.2	域的故障隔离	28
3.6	隔离域的热更新	32
3.6.1	状态保存和恢复	33
3.6.2	状态同步	35
3.7	Moat 适配方法	38
3.8	本章小节	39

4	Moat 的实现	40
4.1	辅助工具	40
4.2	Alien OS	41
4.3	Linux 内核模块	42
4.4	本章小结	43
5	实验评估	44
5.1	实验环境	44
5.2	隔离开销	44
5.2.1	跨域调用开销	44
5.2.2	隔离设备的开销	45
5.2.3	跨域通信开销	48
5.3	故障恢复的开销	49
5.4	热更新	51
5.5	本章小结	54
	结论	55
	参考文献	56
	攻读学位期间发表论文与研究成果清单	64

插图

图 2.1	MPK 工作原理	6
图 2.2	基于虚拟化技术的方案	7
图 2.3	Redleaf 的隔离设计	11
图 2.4	Theseus 的热更新设计	13
图 3.1	Moat 架构示意	16
图 3.2	Moat 的跨域通信	25
图 3.3	域的故障恢复流程	29
图 3.4	有状态域热更新过程	32
图 3.5	无状态域热更新过程	32
图 3.6	Moat 适配方法	38
图 4.1	Moat 在 AlienOS 与 Linux 中的区别	41
图 5.1	null block 性能测试	46
图 5.2	Nvme 性能测试	47
图 5.3	Nvme 写性能测试	48
图 5.4	不同的跨域调用次数和域内工作负载对性能的影响	49
图 5.5	不同频率的故障发生次数对性能的影响	51
图 5.6	调度器的热更新	51
图 5.7	服务暂停时间分析	53

表格

表 2.1	模块隔离和热更新的现有方法总结	5
表 3.1	域基础库核心 API	19
表 3.2	域管理运行时的核心 API	23
表 5.1	跨域调用的时钟周期数	45
表 5.2	故障隔离的内存开销	50
表 5.3	Moat 的热更新服务暂停时间	52

主要符号对照表

GC	垃圾收集
LKM	Linux 内核可加载模块
TCB	可信计算基
RCU	读-复制-更新
IOMMU	输入输出内存管理单元
MPK	用户态内存保护密钥
PKS	特权内存保护密钥
SFI	软件故障隔离

1 绪论

1.1 研究的背景和意义

现代操作系统内核作为计算机系统的核心，承担着管理硬件资源、调度任务和提供基础服务的关键职责。传统单体内核设计通过共享地址空间实现模块间高效协作，却以牺牲安全边界为代价。以 Linux 为例，其庞大的代码库（截至 2022 年，5.11 版本已包含约 3000 万行 C 代码^[1]）中，约 60% 为设备驱动模块，而内存安全问题（如缓冲区溢出、释放后使用）占据了内核漏洞的绝大多数比例。研究统计表明，驱动模块贡献了绝大多数的内核安全缺陷^[2-5]，这些缺陷往往通过复杂的模块依赖链蔓延，引发系统故障。例如，2021 年 Linux 内核的 TIPC 协议栈堆溢出漏洞（CVE-2021-43267^[6]）导致远程攻击者可触发系统崩溃，凸显了模块隔离机制的迫切需求。

内存隔离是模块隔离的关键属性，其目标在于通过硬件或软件手段，防止一个执行单元的错误或恶意行为影响其他单元的内存数据，从而提升系统的安全性和隐私性^[7-8]。同时，内核模块仍然需要高性能的跨模块通信以保证内核的效率。因此，新的隔离方法必须支持低开销的内存隔离。故障隔离对提高内存隔离的有效性有很大的作用^[9-10]。当内核模块在运行时因为违反内存隔离属性或其他原因而出现错误时，模块有责任将错误限制在内部而不蔓延到其它模块中，并且模块需要把发生错误期间分配的资源释放，以避免可能发生的资源泄露和死锁问题。

对内核模块实施热更新是一项富有挑战的任务。支持热更新允许对运行良好的模块进行升级以增加额外的功能，或者对发生过故障的模块进行替换以修复已存在的错误。已有的热更新方法要不只能工作在较小的补丁上^[11-14]，要不就是针对特定的内核模块^[15-19]，缺乏一种有效的机制为不同的模块实现统一的热更新方法。尽管实现模块的隔离并不需要关注热更新，但反过来，实现模块的热更新在一定程度上蕴含了隔离属性，然而，许多工作没有意识到隔离属性和热更新之间的关联性。

操作系统内核模块隔离是一个重要的议题，早在几十年前就被提出^[20-21]。然而，由于历史传承、性能成本考量等因素，Unix、Linux 等主流现代操作系统内核仍然基于 C 语言的宏内核形式，在现有架构下难以解决类型和内存安全问题。到目前为止，相关研究已经有很多成果，例如硬件解决方案^[7-8,22-32] 和软件解决方案^[33-42]，但都存在诸如系统开销等问题难以解决。此外，基于安全语言的隔离相关工作也有一些^[9,43-47]，

但存在较强的限制, 这些传统的安全语言需要运行时以进行垃圾回收, 尽管垃圾回收算法已经进行了高度优化, 但对于具有微秒级延迟的现代高速存储设备来说, 开销依然很高 (在高速网卡设备的典型工作负载下^[48], 最快的垃圾收集语言比 C 语言慢 20-50%)。

近年来, 随着 Rust 语言的兴起及其内存安全性、无需依赖垃圾回收的零成本抽象能力等特性, 为系统编程领域注入了新的可能性。Rust 语言分为两部分: **safe Rust** 和 **unsafe Rust**。**safe Rust** 指的是所有在语言严格的静态检查 (所有权、借用规则、生命周期^[49]) 下都能被证明是安全的操作, 编译器会无条件地确保这些操作不会导致内存错误或未定义行为。**unsafe Rust** 通过 **unsafe** 关键字显式地定义了物理边界, 允许开发者执行编译器无法验证的底层操作 (例如原始指针解引用、内存布局控制或外部函数调用)。尽管 **safe Rust** 具有诸多优势, 但对于操作系统开发而言, **unsafe Rust** 仍然必不可少, 因为 Rust 编译器无法静态保证对各种底层系统资源的操作安全性。因此, 为了能够用 **safe Rust** 编写更多功能, 并充分利用编译器的安全检查——从而减少因误用不安全型代码而导致的漏洞和未定义行为^[50-51], 一种有效的策略是将 **unsafe Rust** 代码限制在最小的作用域内, 并用精心设计的安全抽象对其进行封装。

通过编译时所有权检查与借用规则, Rust 无需垃圾回收即可杜绝数据竞争、悬垂指针等内存错误, 这一特性与内核模块隔离的需求高度契合。**Redleaf**^[45]、**Asterinas**^[47]、**Theseus**^[44]等研究工作开始关注基于 Rust 语言的内核隔离方法或者基于语言机制实现模块的热更新。尽管这些工作探索了 Rust 在提升内核模块安全性和提供更丰富功能方面的潜力。然而, 它们都使用 Rust 构建了自己的新操作系统, 以完全替换现有的商业操作系统内核, 这使得这些解决方案难以应用于现有工作。

总而言之, 现有的解决方案无法在内核模块隔离、模块的热更新以及通用性上找到一个平衡, 因此, 实现一个 (1): 支持模块的高效、灵活和细粒度的内存隔离 (低开销); (2): 支持模块内的故障隔离 (高可靠性); (3): 支持对隔离模块进行组件级别的热更新 (热更新); (4): 隔离和热升级适用于不同的模块, 可以应用在不同的操作系统系统中 (通用性) 的内核模块隔离框架是一个非常值得探索的研究领域。

1.2 本文的研究内容

针对基于 Rust 语言的内核隔离相关工作存在的性能和通用性问题, 本文在 **Redleaf**、**Theseus** 的工作思路的基础上, 提出了一种新的内核模块隔离框架 **Moat**, 基于系统分

解和模块化思想将隔离所需的基础设施与隔离模块的实现进行了解耦，并将 `safe` 代码和 `unsafe` 代码的边界进行了提炼，抽取出一个 TCB 作为隔离的基础设施布置在内核中，其他内核模块可以根据接口基于 TCB 实现基于 `safe rust` 的隔离。其中，TCB 对内核中针对关键资源的 `unsafe` 代码进行了安全性封装，设置域运行时模块和域基础库模块。运行时模块负责系统初始化并在系统运行中管理所有的隔离域 (注册、卸载、热更新)，域基础库模块负责提供安全隔离域实现所需的内核资源安全抽象和用于堆隔离和热更新的各类堆分配器。通过仔细划分系统无关部分和系统相关部分，进而实现对商用宏内核操作系统的良好适配性。归功于良好的模块隔离，本文进一步实现了内核模块的热更新，并针对无状态或有状态的不同特征的内核模块，引入了优化的任务检测方法以减少热更新所需时间。

为了评估所提出的解决方案，本文构建了一个名为 AlienOS 的 Rust 内核，其由 29 个隔离域构成，并基于 MOAT 对 Linux 内核进行了安全重构。实验结果表明，MOAT 的跨域调用速度对比方案快三倍，而吞吐量开销仅为 1% 至 2%。此外，所提出的实时更新策略对比方案实现了更短的服务暂停时间。

本文的主要贡献如下：

1. 本文提出了一种新的隔离架构，用于逐步升级现有商业操作系统内核（例如 Linux）的安全性。所提出的隔离域所需的基础设施与域的实现解耦，并且精心设计了一组接口和安全抽象。
2. 在模块隔离域之上，本文为这些域设计了一种实时更新策略，该策略适用于有状态和无状态模块。Moat 使用 RCU 机制来提高无状态域的更新效率，并基于优化的读写锁方法设计有状态域的实时更新。
3. 基于所提出的隔离抽象模型，本文实现了一个完全符合模块化隔离设计的小型操作系统原型。本文在 Linux 上进一步进行了初步的隔离转换，实现了 Linux 三个内核模块的隔离。

1.3 本文的组织结构

根据本文的研究内容，全文的组织架构如下：

第一章首先介绍了内核模块隔离产生的背景，以及使用不同的方法实现对内核模块隔离的必要性，简要概述了当前比较重要的一些隔离方案，以及其中亟需解决的问

题。最后针对这些问题，介绍了本文的研究工作和组织架构。

第二章介绍了当前内核模块隔离研究的现状，对基于硬件、基于软件、基于安全语言的工作进行了综述和分析，详细介绍了其相关的技术原理以及其可能存在的问题。在总结相关工作的基础上，提出了本文的目标，为后续内容形成理论铺垫。

第三章是本文的重点部分，详细描述了本文提出的内核模块隔离框架：**Moat**。首先，本章介绍了 **Moat** 框架的总体设计思路，以及框架的主要组成部分，从而引出这一章的其它部分。紧接着，对隔离域集合、隔离域基础库、隔离域管理运行时的主要接口和功能进行了分析和描述，明确了这些部分在框架中的作用。有了这些作为基础后，本章对本文的核心目标：隔离和热更新进行了具体分析，包括如何实现内存隔离、隔离模块如何通信、如何利用语言机制实现故障隔离、如何利用隔离属性实现热更新以及相关的性能优化措施等。

第四章是实现部分，主要介绍 **Moat** 框架的代码实现以及其应用。首先，本章介绍了除第三章描述的这些关键基础设施外的额外功能和工具，包括用于简化域实现以及域基础库实现的代码生成宏以及用于简化开发者开发和部署域的自动化工具。然后，本章介绍了一个完全根据 **Moat** 框架实现的 OS: **Alien OS** 的主要组成，并重点介绍了 **Moat** 框架在 **Linux** 内核中的适配情况和对 **Linux** 内核的修改。

第 5 章是实验评估部分。首先介绍了在 **Linux** 和 **Alien OS** 上的测试环境，然后针对 **Moat** 框架的各个机制进行了性能评估和分析，包括跨域调用的微基准测试、隔离的整体性能开销、跨域通信的整体开销、故障隔离的开销等，并对热更新的服务暂停时间和热更新的可扩展性进行了深入分析。

最后为结论部分，对本文的工作进行了全面的总结，并提出对未来工作的展望。

2 研究现状

表 2.1 模块隔离和热更新的现有方法总结

方案	类别	类型和内存安全	内存隔离	故障隔离	热更新	Linux 兼容
LXD ^[7] 、LVD ^[31]	硬件	✗	✓	✓	✗	✓
Sel4 ^[40] 、XFI ^[35]	软件	✗	✓	✓	✗	✗
XRP ^[38] 、ksplit ^[39]	软件	✗	✓	✓	✗	✓
Verve ^[52] 、Biscuit ^[53]	安全语言	✓	✗	✗	✗	✗
Tockos ^[54] 、Asterinas ^[47]	安全语言	✓	✗	✗	✗	✗
Theseus ^[44]	安全语言	✓	✗	✓	✓	✗
Redleaf ^[45]	安全语言	✓	✓	✓	✗	✗
Moat	安全语言	✓	✓	✓	✓	✓

在计算机发展早期，受限于硬件性能，操作系统开发者在效率与安全性难以兼顾时往往优先考虑效率。随着新型 CPU 架构^[55-56]及高速设备（如 NVMe 固态硬盘^[57]、智能网卡^[58]）的出现，硬件性能已不再是内核开发的主要瓶颈。过去数十年间，大量用户态与内核项目探索了三种实现内核模块隔离与热更新的技术路径：(1) 硬件虚拟化及专用硬件机制^[7-8,22-32]；(2) 基于软件的隔离方案^[33-42]；(3) 基于语言的类型系统与安全机制^[9,43-47]。

本章将依据本文的主要研究内容，概述当前的研究现状与相关理论。本章首先分别介绍几种常见的内核模块隔离方法，然后介绍热更新相关的研究。最后针对已有的工作进行总结并提出本文的目标，表 2.1 总结了主要工作与 Moat 的比较。

2.1 基于硬件机制的隔离

根据谷歌^[59]、微软^[60]与苹果^[61]的报告，内存安全问题占已发现漏洞的 60%–70%。由于历史性能优先策略导致这些系统中广泛使用不安全的 C/C++ 代码，进一步加剧了风险。解决内存安全问题的直接方案是实施隔离机制。从功能层面看，隔离主要包含两个核心维度：内存隔离（Memory Isolation）：作为基础机制，确保不同执行单元（如进程、内核模块或子系统）间的内存访问互不干扰；故障隔离（Fault Isolation）：将

x86_64 Memory Protection Keys (MPK) 工作原理

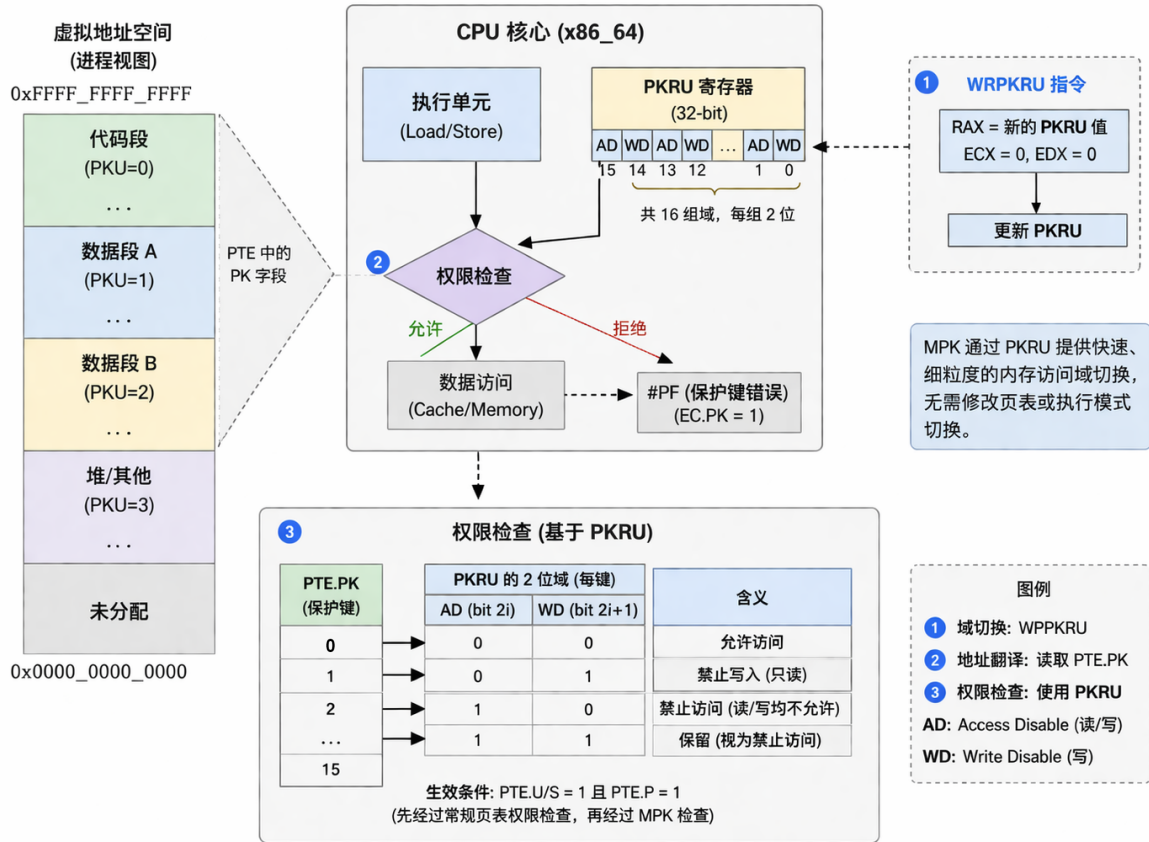


图 2.1 MPK 工作原理

局部故障严格限制在发生点，其实现通常以内存隔离为基石。这些机制对提升系统安全性与稳定性至关重要。

尽管硬件级内存隔离机制（如虚拟内存与特权分级）为用户空间程序提供了基础保护，但用于地址空间隔离的进程抽象会引入过高开销。新兴技术如内存标记（MPK^[62]）与指针认证（ARM MTE^[63], ARM PAC^[64]）能实现更细粒度的进程内隔离。这些技术探索了以内存标签和功能为中心的细粒度隔离的想法，内存保护密钥（MPK 2.1）是 Intel 在 SkyLake CPU 中引入的一种新的隔离机制。MPK 允许通过使用 4 位保护密钥（保存在页表条目的未使用位中）标记各个页面，在单个地址空间内强制实施隔离。受限于 MPK 只能作用在用户态内存页上，^[8,26,32]等使用 MPK 在内核中进行隔离的方案针对的是 unikernel 形态的单特权级操作系统。与 MPK 类似，ARM SoC 引入了对内存标记扩展 (MTE) 的支持，允许将地址空间划分为 16 字节区域，并使用 16 个标记之一进行着色，配合加密指针身份验证 (PAC) 技术，可以提供高效的隔离。不过，由于只能使用较小数量的颜色对内存进行标记，这些机制也限制了在具有大量

模块的系统上进行隔离的可能性。

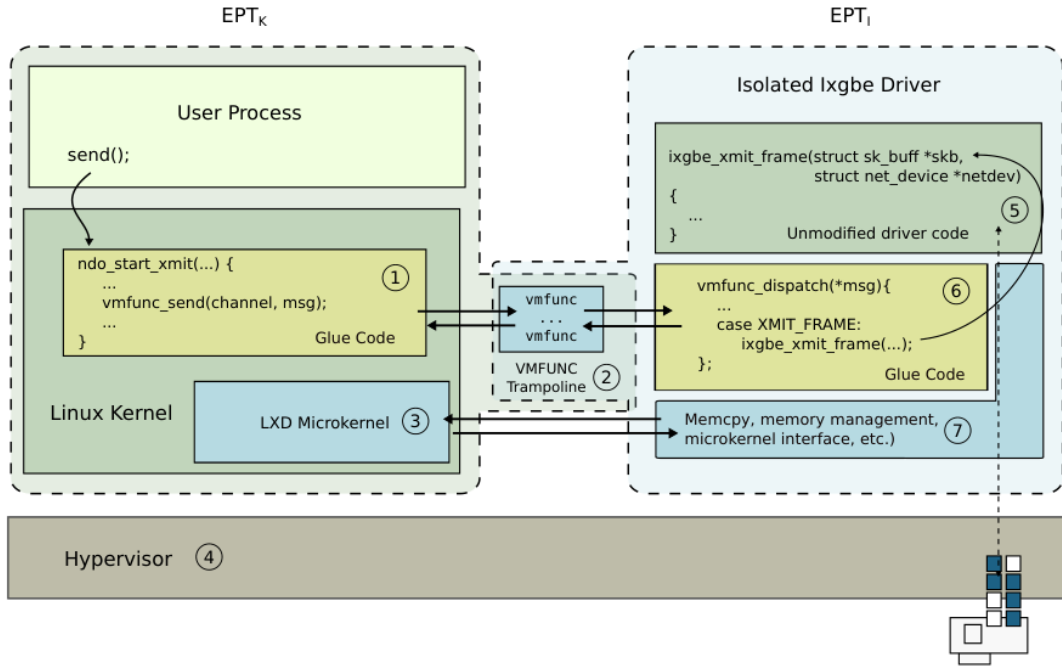


图 2.2 基于虚拟化技术的方案

为降低易出错非核心模块（如驱动程序）的影响，基于硬件的隔离方案^[7-8,22-32]通过硬件特性直接在物理层面实现对内核模块的隔离，增强安全性和性能。例如，Intel VT-x^[65]和 AMD-V^[66]等硬件虚拟化技术允许虚拟机管理器高效地运行多个虚拟机，并在硬件层面隔离彼此的资源（图 2.2）。可信执行环境（TEE）技术如 ARM TrustZone^[67]提供安全的硬件环境，用于执行敏感操作或存储关键数据，防止恶意软件或管理员的干扰。此外，IOMMU（输入输出内存管理单元）通过控制外设对内存的访问，进一步提升了隔离的粒度和安全性。然而，这些技术对实施普通内核模块的隔离过于昂贵，由于单个模块通常使用一个单独的虚拟机或者隔离域运行，随着隔离的模块数量逐渐增多，这些方法也无法进行扩展。x86 架构下，还引入了与 MPK 对应的技术 PKS（Protection Keys Supervisor），运行在内核态下的内存标记技术，但由于 16 个标记的限制，它也缺乏扩展性。

2.2 基于软件方法的隔离

尽管现代内核采用地址空间布局随机化^[68]、栈溢出保护^[69]等众多工具检测和缓解常见内存安全漏洞，但这些机制均不是面向内核模块设计的。

基于软件的隔离机制^[10,33-39]通过在操作系统中建立逻辑约束实现模块分离。经典

方案软件故障隔离（Software Fault Isolation, SFI）通过插入边界检查指令限制内存访问与代码执行，强制建立安全边界，阻止不可信代码访问未授权内存区域或执行恶意操作。SFI 的核心机制是对内存地址和控制流的动态检查，通过屏蔽地址或限制跳转目标，将代码的行为限制在预定义的合法范围内，同时通过静态验证器在加载代码时确保其符合安全策略。虽然 SFI 在支持多语言隔离、降低隔离开销与跨模块通信成本之间实现了有效平衡^[70-71]，但其硬件无关的实现方案仍存在显著性能损耗^[25]。这些局限源于两个根本性约束：

- 语义盲区：SFI 依赖编译器插件生成安全检查，但编译器缺乏对程序逻辑的语义理解，只能机械地为所有潜在代码路径插入检查指令，从而导致性能下降；
- 人工依赖：需要开发者手动指定待保护模块与函数。

微内核架构也是一种软件隔离方法。微内核架构^[40-42]通过将操作系统的核心功能极度精简，仅保留进程管理、内存管理和进程间通信（IPC）等基本功能，其余诸如文件系统、驱动程序、网络协议栈等服务被移至用户态运行，从根本上改变了传统单体内核的设计。这样的设计实现了明确的模块边界，每个服务运行在独立的地址空间中，通过受控的 IPC 进行通信，这种强隔离不仅提高了系统的安全性，也增强了模块的可靠性——一个服务的崩溃通常不会直接导致整个系统的瘫痪。典型的微内核操作系统如 seL4^[40]和 MINIX 3^[41]展示了高安全性和可靠性的设计理念，这些系统在航空、医疗等安全关键领域得到了广泛应用。然而，微内核的隔离机制也引入了一些缺点。由于服务之间的频繁通信需要通过用户态和内核态的多次上下文切换，通信开销较高，导致性能上的损失。除此之外，子系统交互方式的彻底改变导致现有的大量内核组件无法被复用，而在用户态提供这些组件所依赖的运行环境需要大量的工程工作，还面临内核子系统接口变化的问题。与微内核类似的技术还有用户态用户驱动，比如 DPDK^[72]、Fuse^[73]等。

2.3 基于传统安全语言的隔离

操作系统模块隔离与热更新的研究持续演进。随着编程语言的进步，系统级语言（如 C#、Go 和 Rust）将高性能与内存安全相结合，正从基础层面解决内存安全问题。根据谷歌报告显示，随着开发转向内存安全语言，Android 系统中内存安全漏洞占比在六年内从 76% 降至 24%^[74]。这些语言为构建高效、轻量级且细粒度的隔离机制开

辟了新路径——现代编程语言的类型系统与安全保障，可在不损失性能的前提下实现上述目标。

大多数安全语言依赖垃圾回收机制 (如 C#^[75]和 Go^[76])，相比利用所有权^[77]和线性类型系统^[78]实现类型和内存安全的 Rust^[79]语言，它们无需手动分配/释放内存，也不需要关注所有权和生命周期等复杂的编程概念。尽管优势明显，但基于垃圾回收的安全语言在实现低开销的隔离方面仍有困难。

Verve^[52]提出并实现了操作系统 Verve，这是一种使用类型化汇编语言 (TAL) 和自动化定理证明技术验证类型安全和内存安全的操作系统。Verve 采用分层结构设计，其中低层的 Nucleus 负责与硬件和内存的交互，并通过 Hoare^[80]逻辑进行正确性验证，而高层的内核和应用则使用安全的 C# 代码编写，并编译为 TAL 后自动验证其安全性。通过这种设计，Verve 实现了对系统各个组件的静态验证，包括运行时系统、设备驱动和应用程序的安全性，从而在操作系统中消除了缓冲区溢出和内存悬挂等常见低级错误。Biscuit^[53]探讨了使用高级语言 Go 编写 Posix 内核的可行性，其开发充分利用了 Go 语言的类型安全、垃圾回收和并发机制。

Verve^[52]和 Biscuit^[53]使用基于垃圾回收的语言重新实现操作系统，这些工作证明了现代高级语言可以从底层设计上降低操作系统内核发生的错误并在高层简化对操作系统的形式化验证工作，然而，由于这些语言通常不提供低级硬件交互的接口，这导致需要修改其运行时，而且，其垃圾回收器存在不可预测的暂停时间，这些缺陷造成了无法在商用操作系统中得到支持。

2.4 基于 Rust 语言的隔离

Rust 是一门相对年轻的系统编程语言，诞生于 Mozilla 研究院，其核心目标是解决一个长久困扰系统编程领域的难题：如何在不牺牲性能的前提下，实现内存安全和线程安全？传统上，像 C 和 C++ 这样的语言提供了极高的性能和对硬件的精细控制，但它们依赖程序员手动管理内存（如分配和释放），这极易导致诸如空指针解引用、缓冲区溢出、数据竞争等严重错误，这些错误往往是安全漏洞和程序崩溃的根源。另一方面，拥有垃圾回收 (GC) 机制的语言（如 Java, Go, Python）虽然大大减轻了内存管理的负担并提高了安全性，但 GC 带来的运行时开销和不可预测的停顿时间，使得它们在某些对性能和资源控制有严苛要求的系统编程场景（如操作系统内核、游戏引擎、嵌入式设备、高性能网络服务）中显得力不从心。

Rust 致力于成为一门安全、并发、高效的语言，同时保持实用性和高性能。它不依赖垃圾回收，而是通过一套独特且强大的编译时所有权系统与借用规则来达成其安全目标。这套机制是 Rust 最核心的创新和标志性特征。

所有权的概念是 Rust 的基石。在 Rust 中，每一个值在任意时刻都有一个明确的所有者（通常是一个变量）。这个所有权遵循几条关键规则：首先，一个值只能有一个所有者。其次，当所有者（变量）离开其作用域时，它所拥有的值就会被自动、确定性地清理（释放内存或其他资源），这个过程称为“Drop”。最后，值的所有权可以在变量之间移动（Move），移动发生后，原来的变量将失效，不能再被使用，从而避免了意外的“双重释放”或“悬垂指针”。这种移动语义默认发生，确保了资源的唯一归属。这与许多语言默认进行浅拷贝（可能留下悬垂指针）或深拷贝（可能带来性能开销）不同。当确实需要复制堆上的数据时，Rust 要求显式调用克隆（Clone）方法。

然而，如果只有所有权和移动，编写复杂程序会非常受限，因为频繁地移交所有权会导致代码难以组织。因此，Rust 引入了借用的概念。借用允许你临时引用一个值，而不需要获取其所有权。引用分为两种：不可变引用和可变引用。不可变引用允许你读取数据但不能修改它，并且可以同时存在多个不可变引用指向同一个数据。可变引用则允许你修改数据，但有一个极其严格的限制：在同一个作用域内，对同一块数据，要么只能存在一个可变引用，要么只能存在多个不可变引用，二者绝对不可共存。这个规则由编译器在编译时严格检查，被称为借用检查器。

为了实现内存安全和零成本抽象，Rust 引入了生命周期的概念。生命周期是 Rust 编译器用来跟踪引用有效作用范围的一种标注。它确保引用在整个使用期间都指向有效的内存。虽然编译器通常能自动推断生命周期，但在某些复杂场景（尤其是函数签名或结构体定义中涉及引用时），需要开发者显式标注生命周期参数，以向编译器提供必要的信息。

为克服传统安全语言的局限，部分研究转向采用安全语言 Rust——其性能与 C 语言相当——来开发兼具高性能与强安全保障的新系统。Redox^[81]、RustyHermit^[82]及 Drone^[83]等项目虽在不同架构上实现了 Rust 内核，但仅将其作为更安全的 C 替代品。TockOS^[54]与 Asterinas^[47]则更进一步：通过将内核划分为不安全世界与安全世界，其中小型不安全世界可利用 Rust 的 unsafe 特性执行面向硬件的底层操作，并向安全世界暴露安全抽象接口；而安全世界完全采用 safe Rust 编写，从而保留 Rust 的安全保障。

鉴于 Rust 的安全与性能优势，主流操作系统已开始将其集成至内核：Linux^[84]与 Windows^[85]正逐步引入 Rust 代码，旨在利用其类型安全与内存安全特性减少内核缺陷并提升系统稳定性。这些系统通过混合 Rust 与现有 C/C++ 代码库，实现了向更安全编程实践的渐进式迁移。然而，此类系统尚未探索 Rust 在类型与内存安全之外的深层能力，如模块隔离与热更新机制。

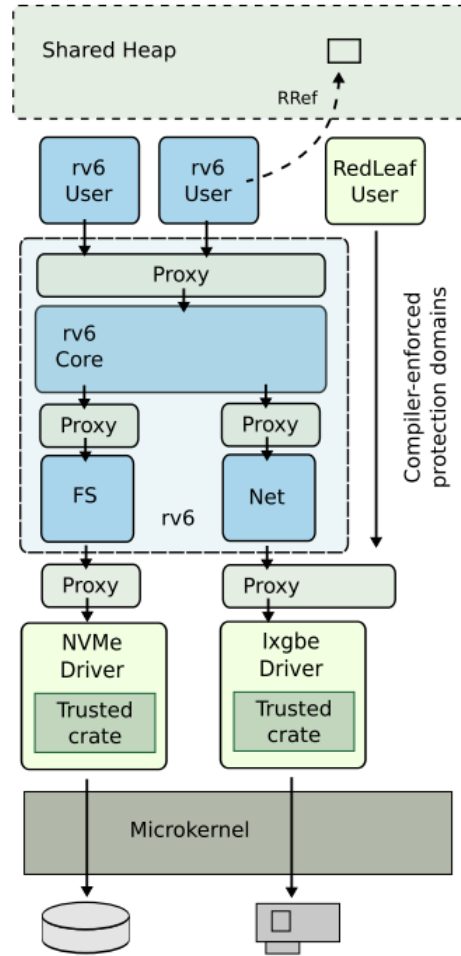


图 2.3 Redleaf 的隔离设计

Redleaf^[45]是一个使用 Rust 开发的微内核，旨在探索语言安全性对操作系统组织结构的影响，尤其是在实现细粒度隔离方面的潜力。不同于依赖硬件地址空间的传统操作系统，RedLeaf 利用 Rust 的类型和内存安全特性实现隔离。这种设计避免了昂贵的硬件隔离机制，使得轻量级隔离成为可能，并且降低了性能开销。RedLeaf 的核心抽象是语言支持的隔离域（language-supported isolation domain），这些域作为独立的隔离单元，可动态加载和安全卸载，并且崩溃时不会影响其他域的执行。与之前利用语言机制实现子系统隔离的系统^[9,43,46]类似，RedLeaf 中隔离的实现依赖以下原则：堆

隔离、可交换类型、所有权追踪、接口验证和跨域调用代理。尽管 `redleaf` 在使用 `Rust` 实现高效、细粒度的隔离子系统上已经迈出了一大步，这并没有发挥出 `Rust` 语言的所有能力。

首先，`Redleaf` 缺乏对 `safe` 和 `unsafe` 边界的划分，与 `TockOS`^[54] 和 `Asterinas`^[47] 类似，`Redleaf` 强制要求隔离域禁止使用 `unsafe` 代码，然而其大多数域都依赖于域相关的可信库，而可信库中大量的 `unsafe` 代码破坏了 `Rust` 语言提供的安全保证，使得整个系统中完全由 `safe` 代码构成的部分比例大大减少。其次，`Redleaf` 依靠状态检查点机制来实施故障隔离，然而，仅仅靠检查点状态的保存和恢复无法正确释放由故障导致的资源泄露的，如果没有将检查点到故障点之间的分配的资源释放掉，即使可以再次对已经发生故障的域发起请求，域仍然处于一个不正确的状态。最后，`Redleaf` 没有意识到，对子系统的隔离可以带来更多的扩展功能，比如实现子系统的热升级。

为了应对现有的基于 `Rust` 的操作系统开发中不正确的 `unsafe Rust` 使用，`Asterinas` 提出了一种新的操作系统架构，称为 `framekernel`，它实现了 `Rust` 实现内核内部特权分离的全部潜力，确保了 `TCB` 的最小化和可靠性。在 `framekernel` 架构中，整个操作系统驻留在一个单一的地址空间中，并在 `Rust` 中实现。内核在逻辑上分为两个部分：特权操作系统框架（类似于微内核）和非特权操作系统服务。只有特权框架才允许使用不安全，而去特权的服务必须完全用安全的 `Rust` 编写。作为 `TCB`，特权框架将所有低级的、面向硬件的不安全操作封装在安全 `api` 后面，被剥夺特权的操作系统服务使用这些 `API` 实现各种操作系统功能。`Asterinas` 在对 `Rust` 的 `safe` 和 `unsafe` 边界划分上向前迈进了一大步，提出了许多值得借鉴的设计思路。

上述这些工作探索了现代安全语言在提高内核安全性上的潜力，进而提供更丰富的功能，比如内核模块隔离、安全内核扩展等。然而，它们目前依然存在通用性的关键问题尚未解决，即都是利用 `Rust` 语言重新打造了各自的全新操作系统。其中，`Redleaf` 没有考虑其隔离方案在主流宏内核操作系统上的适配，导致适用性受到影响；而 `Asterinas` 则是为 `Linux` 应用提供匹配的系统调用。

2.5 热更新

热更新是一种在无需重启系统的情况下对内核模块进行更新的技术，分为功能级别^[11-14]和组件级别^[15-19]两种主要方案。`kpatch`^[11]、`livepatch`^[12]等功能级别的热更新允许开发者通过补丁形式直接修改或替换内核中的特定函数，完成漏洞修复或功能增

强，而无需重新启动系统。功能级别的优点在于其实现相对简单，只需将变更限定在特定的功能模块内。然而，其局限性也非常明显，即只能在变更对系统整体结构影响较小的情况下使用，对于涉及接口修改或跨模块依赖的大型更新，功能级别的热更新显得力不从心。相比之下，组件级别的热更新旨在对整个模块进行全面替换，通过保存旧版本的运行状态，并在新模块加载后迁移状态来实现无缝切换。组件级别的热更新的 3 个难点在于（1）：维护组件的状态，在更新后，组件以更新前的状态继续服务；（2）：尽可能降低服务暂停时间，这会显著影响那些延迟敏感型应用^[86-87]（3）：适用于不同的模块。操作系统模块之间复杂的关系和内部复杂的状态导致已有的支持组件级热更新的方案往往不能兼顾这 3 点。比如，PlugSched^[15]支持对调度器模块进行热更新并具有较短的服务暂停时间，却无法直接迁移至其他模块。

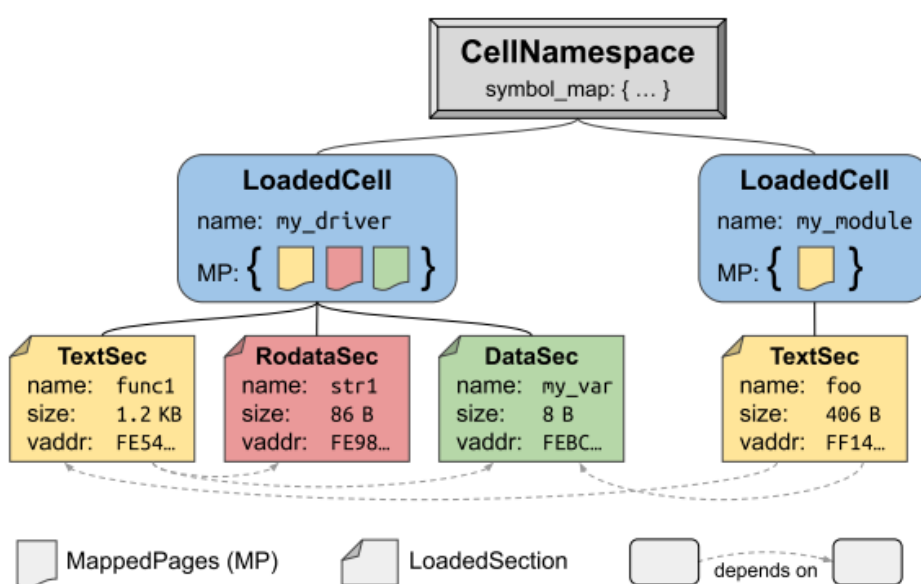


图 2.4 Theseus 的热更新设计

Theseus^[44]是另一个从零开始使用 Rust 编写的实验性操作系统，它重新思考了操作系统的结构和状态管理问题。其聚焦于解决现代操作系统中普遍存在的状态泄漏问题，这种问题导致模块间的强耦合，严重阻碍了系统的演进、容错性和灵活性。Theseus 的设计核心在于通过明确划分操作系统模块边界、最大化利用语言特性以及消除模块间状态泄漏，构建一个可以动态演进和高效容错的系统。Theseus 中的每个模块被称为“cell”，它们是运行时独立且可动态加载或替换的单元。这种设计确保了系统在运行时的灵活性，支持模块的实时热插拔和无中断更新。Theseus 通过 Rust 的内存安全

和类型安全特性，将许多传统由操作系统负责的任务（如内存管理和资源回收）转移到编译器层面。这种“内在化设计”（Intralingual Design）减少了操作系统本身需要维护的状态量，简化了状态管理。特别是，Theseus 实现了无状态的模块交互，使得模块更新或恢复时无需担心状态一致性问题。此外，通过模块化的设计理念，Theseus 使得故障模块可以被独立隔离并恢复，从而提升了系统的容错性。

不幸的是，由于一贯的特权级分离设计，现代商用操作系统内核子系统中保存了大量用户任务的状态，各个子系统之间依靠复杂的状态数据进行交互，为了实现 Theseus 提倡的无状态设计方法，商用操作系统内核需要进行彻底的推翻和重构，这在当前显然还不能实现。

2.6 总结和目标

在这些工作的启示的基础上，Moat 旨在实现一个通用的内核模块隔离和热更新框架，以探索安全语言 Rust 对操作系统安全性、可靠性和可扩展性的促进作用。

性能优先 Moat 将性能作为首要考量，明确内核隔离与热更新机制不得损害系统效率。受 Redleaf^[45]启发，Moat 采用基于语言的隔离方案实现模块隔离，从而避免了硬件方案固有的上下文切换与特权级转换开销，并支持模块间低开销隔离。为了进一步降低 Redleaf 中隔离模块之间的通信开销，Moat 需要避免引入额外的状态检查点机制。

通用隔离架构 Moat 的设计并非为了实现隔离目标而设计一个全新的操作系统，而是将隔离和热更新的基础设施与模块实现解耦，使现有操作系统能够以最小的调整重复使用核心机制。其与既有内核组件协同运作，例如允许基于 Rust 的隔离域与传统可加载内核模块（LKMs）并存。开发者可根据安全需求灵活选择隔离域或传统 LKM。此外，Moat 提供工具链支持以简化隔离域的开发、编译与部署流程，从而促进原有的模块向隔离域逐渐迁移。

划分安全和不安全的边界 由于采用了基于语言的隔离方法，Moat 不可避免地需要处理 Rust 语言中的不安全子集。在分离实施隔离所需的基础设施和模块实现的基础上，Moat 需要定义安全代码和不安全代码之间的清晰边界，从而解决 Redleaf^[45]中大量的不安全代码对整个系统的影响问题。safe 和 unsafe 的边界划分不仅可以让被隔离的模块从安全的 Rust 代码中受益，还可以让不安全的代码保留在较小的范围内，进而允许对其进行必要的安全审查，从而增强可靠性。

热更新支持 为了解决模块状态难以管理的问题，在模块隔离的基础上，Moat 提供了一个通用的内核模块热更新框架。该框架提供了必要的数据结构和接口来保存和恢复模块状态，并在模块更新时自动迁移状态数据，从而实现对各种隔离内核模块的热更新。与 Theseus^[44]不同，Moat 不要求模块是无状态的。此外，与 kpatch^[11]（仅限于函数级更新）和 Plugsched^[15]（调度器特定）不同，Moat 提供了一个统一的热更新方法，适用于任意模块——这需要足够的表达能力和通用性。而为了解决模块热更新导致的服务暂停问题，Moat 同样借助隔离带来的边界划分，提供高效的升级时机检测和快速的状态同步机制，以最小化服务停机时间。

2.7 本章小结

本章系统梳理了内核模块隔离与热更新技术的研究现状。现有工作主要围绕三大技术路径展开：基于硬件虚拟化或专用硬件的隔离方案虽能实现强隔离，但存在性能开销大、扩展性差的问题；基于软件的隔离方法（如软件故障隔离和微内核架构）虽更具灵活性，却往往引入显著的性能损耗或需彻底重构系统，难以兼容现有生态；基于安全编程语言的方案（如使用 C#、Go 等语言）虽能提升内存安全，但其运行时垃圾回收机制难以满足内核开发对性能和实时性的严苛要求。

近年来，随着 Rust 语言在系统编程领域的兴起，基于 Rust 的操作系统研究（如 Redleaf、Theseus、Asterinas）展示了利用其所有权模型和类型系统在实现内存安全、细粒度隔离乃至无状态热更新方面的巨大潜力。这些工作证实了语言级安全机制可在不依赖硬件的情况下高效实现内核模块隔离。然而，它们多为从头构建的全新系统，其隔离与更新机制与特定系统架构紧密耦合，缺乏通用性，导致难以直接应用于 Linux 等现有主流操作系统进行渐进式改进。

此外，在热更新方面，现有方案如 kpatch 等功能级更新灵活性不足，而 Plugsched 等组件级更新又高度特化，缺乏通用、高效且能维护状态一致性的通用方案。Theseus 提出的无状态设计虽理想，但要求推翻现有内核的状态管理模型，实用性受限。

在此研究背景下，本文工作的目标是设计并实现 Moat 这一通用框架。它旨在克服现有方案的局限，核心目标是利用 Rust 的语言安全特性，在不牺牲性能的前提下，为现有主流操作系统提供一个兼具高性能、通用性、清晰安全边界的内核模块隔离与热更新解决方案。Moat 力求在隔离机制与业务逻辑之间实现解耦，使其能灵活适配不同系统，并支持对已有模块进行渐进式的安全加固与动态更新。

3 Moat 内核模块隔离框架设计

在总结和分析了已有工作的基础上, 本文创新性地提出了 Moat, 一个用于实现内核模块隔离的框架。该框架的设计有效地实现了在第二章中讨论的目标, 即使用安全语言实现模块隔离以确保高性能, 将机制和策略进行解耦以实现通用性和可复用性, 通过区分安全和不安全的代码以明确安全边界以及基于模块隔离基础实现模块的热更新。本章的剩余内容将详细介绍 Moat 的核心设计与适配方法。

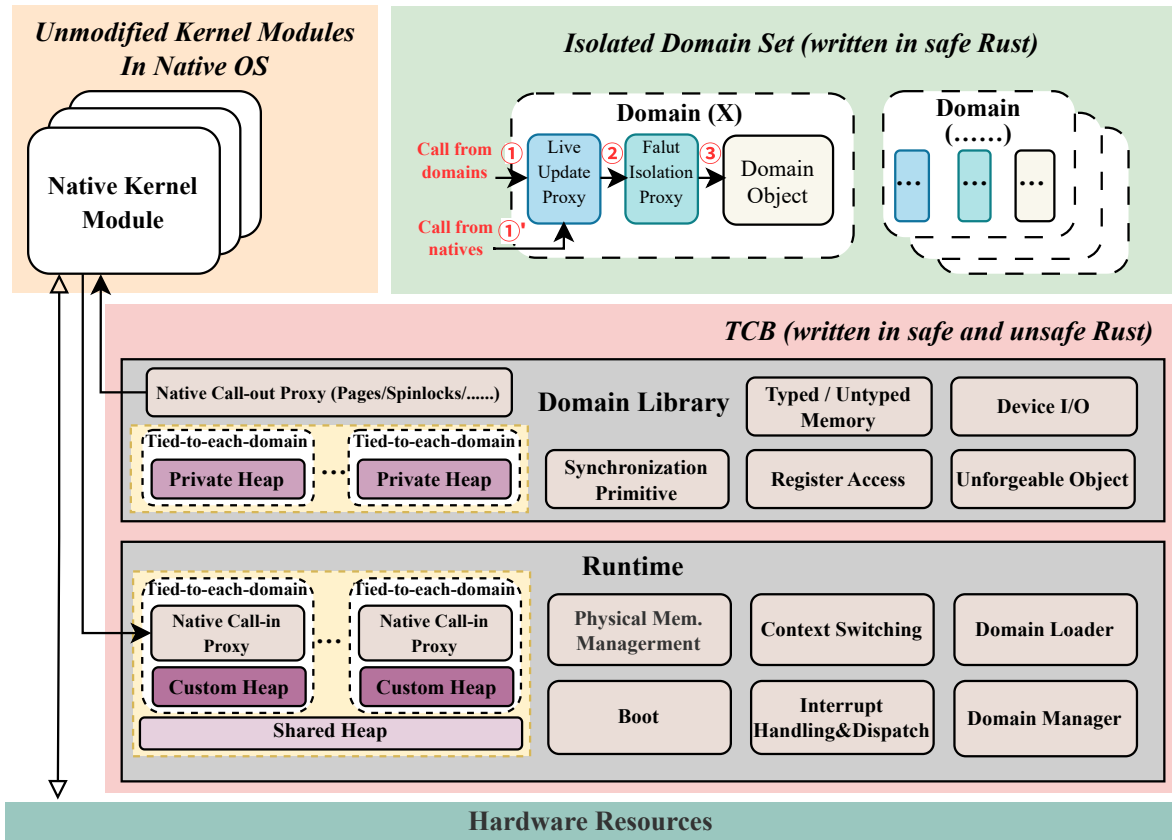


图 3.1 Moat 架构示意

3.1 内核模块隔离的总体设计思路

为实现 Linux 等商用操作系统中的内核模块隔离, Moat 的核心设计思想是利用 Rust 语言构建可复用的 TCB 子系统, 封装不安全操作并为隔离模块提供标准安全接口。若需对现有模块进行隔离化改造, 开发者需根据 TCB 接口使用 safe Rust 重写模块, 改造后的模块可与未修改的内核模块共存。在此框架下, 操作系统可通过渐进式改造实现隔离化, 使模块获得 Moat 提供的安全性与热更新能力。图 3.1展示了 Moat

架构，其将内核划分为两大组件：（1）TCB：由域管理运行时模块与域基础库模块构成，集成所有 `unsafe Rust` 代码以执行系统必要操作（如资源分配）。域基础库模块为底层操作提供安全抽象，运行时模块作为最小化运行时环境提供基础系统服务。二者共同构成隔离与热更新的基础设施，使开发者专注于隔离域实现而无需处理底层细节。内部实现上，这两个子模块均分解为系统相关与系统无关部分。（2）域集合：由完全基于 `safe Rust` 实现的隔离域组成的集合。

通过对系统功能的分解，Moat 将一个具有隔离属性的内核系统划分为隔离域集合、隔离域基础库和隔离域管理运行时，从而将隔离的机制和策略解耦。隔离域集合定义了隔离单元-隔离域，并描述了其内部约束与外部特征。域基础库的划分则将系统中不安全的部分从域中分离出来，从而允许隔离域可以完全由 `safe` 的 `Rust` 代码构成。Moat 将那些对开发者来说非常危险的操作，封装在一系列安全的类型背后，并向域开发者提供安全的接口。为了最小化由不安全代码构成的域基础库，域基础库只处理那些必须的不安全操作，而更高级的抽象则由域开发者实施。Moat 将对域的管理和系统的基本初始化功能划分到域管理运行时子模块中。为了在不同的系统之间适配 Moat，域基础库和域管理运行时子模块都由更小的组件组成。

在域基础库的安全抽象之上，Moat 利用堆隔离的思想来实现模块的内存隔离。此外，Moat 支持域的故障隔离，从而将域内部的故障隔离在单个域内部，而不导致其它域出现错误。内存隔离是故障隔离的基础，因为内存隔离确保了发生错误的域无法破坏其它正常域中的内存。为了实现故障隔离，Moat 利用了 `Rust` 语言内置的异常处理机制，在发生故障后，通过执行堆栈展开，所有在故障期间分配的资源得以释放，并最后向调用者返回错误。故障代理进一步简化了故障隔离的实现，使得开发者可以透明地使用该功能。

隔离属性在另一个角度上促进了对模块进行热更新。为了实现一个通用高效的热更新方案，Moat 引入了两个独特的机制。首先，Moat 将域分类为无状态域和有状态域，并为有状态域引入了一个自定义堆，用于存储域内部的状态数据，这允许在域更新前后维护域状态数据的一致性并轻松地迁移。其次，为了降低模块更新对用户程序的影响，Moat 针对无状态域和有状态域实行不同的状态同步方法。无状态域使用 RCU 机制避免了服务暂停，有状态域使用了优化的动态锁使能方法，配合域的接口属性，可以高效地进行任务检查和快速热更新。与故障隔离类似，热更新代理的引入屏蔽了热更新的实现细节，为域开发者提供一个透明的接口对象。

3.2 隔离域集合

在 Moat 中，绝大多数内核功能以隔离域形式实现为动态管理的内核模块。这些域支持运行时加载、卸载或热更新。作为软件定义的隔离单元，它们通过 Rust 的类型与内存安全机制（而非硬件方案）实现隔离。每个域通过域接口（Domain Interface）对外提供服务，在内部，隔离域依赖域基础库中的安全抽象，域的所有的代码完全由 `safe rust` 编写。每个域都遵守相同的内存管理方式，其包含三类核心堆：（1）私有堆（`private heap`）：用于隔离域内部内存分配（§3.5.1）；（2）共享堆（`shared heap`）：支持跨域通信（§3.5.1）；（3）自定义堆（`custom heap`）：存储域的状态数据（§3.6.1）。跨域通信通过域接口的函数调用实现。

域是被独立编译并被动态加载的，这有点类似于 Linux LKM，但又与 LKM 有明显的差异。Moat 使用 Rust 的 `trait` 系统为每一类域定义一个 `trait`（一组函数的集合）从而提供一个接口的抽象，这基于对系统的观察：内核中的模块通常以一组函数请求其它模块服务，并导出一组函数给其它模块服务。每个域需要实现其特定的 `trait`。在实现层面上，域被强制从 `unsafe` 代码中解放出来，因为 Rust 的类型安全和内存安全无法在 `unsafe` 代码中得到有效验证，因此域的依赖项只能包含 Rust 的 `core` 和 `alloc` 安全集合以及域基础库中的安全抽象。换句话说，Moat 的隔离域具有在实现时定义明确的边界，并在运行中持续存在。域对外只暴露明确的接口，内部实现细节对外隐藏，域之间的交互通过接口调用实现。

私有堆和共享堆的建立是隔离属性的基本要求。对于每一个独立可加载的域来说，其内部所需的内存分配除了来自栈上的分配之外，还需要一个堆进行支撑，如果所有的域都共享同一个堆进行分配，当域发生故障时，就无法对域所分配的内存资源进行快速地回收，尽管可以通过维护内存分配的元数据来识别内存资源的所属关系，这会带来严重的内存开销并且仍然没有加速其回收过程。相反，为每一个域建立专属的私有堆，在对域进行更新时，允许直接对旧的域的私有堆进行整体回收。

在满足域内部的内存分配情况下，需要考虑隔离域之间的交互情况。这和域内部不同功能之间交互不同，一个域内部，可能包含不同的功能模块，而这些模块是直接使用函数调用来进行交互的，这些函数调用传递的数据都来自自己私有的堆栈上，不会传递到其它域中。域间的函数调用则有不同的限制，由于需要确保域间的内存隔离，不能在域间调用上传递域的私有数据，因此，为所有域提供一个共享的堆是一个理想

的选择，共享意味着其内存是可以由不同的域进行访问的。

自定义堆有点类似于私有堆，但它们执行不同的任务。自定义堆将域的状态数据与其它数据的存储区分开来，从而实现域更新的状态一致性维护。因此，域的开发者在决定支持域的热更新能力时，有责任确保域的状态数据和普通数据被正确区分，并且状态数据的分配被正确定向到自定义堆上。

图 3.2展示了域的基本形式，以 Domain A 为例，在其内部功能实现中，它通过使用 `Vec::new()` 直接在私有堆上分配堆内存，这和普通的 rust 程序是一样的。在调用 Domain B 的功能前，A 通过 `Dvec::new()` 在共享堆上分配了内存，并通过 B 的域接口将数据移动到 B 内部。A 为了保存其自身的状态数据，则通过 `Arc<dyn VfsInode, CustomHeap>` 定义了数据将会被存储到自定义堆上。代码 8展示了域接口的定义，可以看到，在需要共享或传递数据时，域的接口中都明确定义了需要使用共享堆的数据结构。

3.3 隔离域基础库

表 3.1 域基础库核心 API

APIs	描述
<code>CoreFunction::*</code>	调用域管理运行时中的功能
<code>HeapAllocator::alloc/dealloc</code>	分配私有堆数据
<code>SharedHeapAlloc::alloc/dealloc</code>	分配共享堆数据
<code>storage::insert/get/remove</code>	存储状态数据
<code>Mutex/SafeIORegion/SafePtr/BasicFrame/...</code>	内核资源的安全抽象

3.3.1 基础库的基本功能

域基础库中的安全抽象是建立在对域的分析基础上的：在使用 Rust 进行操作系统开发时，针对低级内核资源的一些操作不得不使用 `unsafe Rust` 进行实现，但是域不允许使用 `unsafe Rust`，同时 Rust 语言中的内置类型也不足以表达这些资源。为此，在域基础库中，Moat 将编译器检查的不变量（无缓冲区溢出，无释放后使用）扩展到对低级内核资源的操作中，让这些资源被无缝地纳入到 Rust 提供的强大静态类型安全中，这是通过对低级内核资源和相关联的操作进行安全抽象来实现的。对低级内核资

源的安全抽象通常是与系统相关的，但是用于实施内存隔离的私有堆和共享堆以及用于实施热更新的自定义堆等基础设施是与系统无关的，因此，Moat 进而将域基础库分解为较小的组件，以允许 Moat 应用到新的系统中时重用这些基础设施。

针对域基础库的系统无关部分，Moat 提供通用设计与类型抽象并暴露安全接口。表 3.1 列出了域基础库的主要 API，包括私有堆、共享堆、自定义堆的分配与释放、状态数据的存储与检索，以及针对互斥锁、安全指针等内核资源的安全抽象。域基础库作为模块化组件实现，核心目标有两部分：

- 安全与不安全边界：将所有需要 `unsafe Rust` 的底层操作封装为内存/硬件访问的安全接口（系统相关）。
- 堆隔离机制：为隔离域提供私有堆、自定义堆与共享堆（系统无关）。

`CoreFunction::*` API 调用域管理运行时的接口访问系统资源。`HeapAllocator` 与 `SharedHeapAlloc` API 分别处理私有堆与共享堆的内存分配。这些 API 开发者不直接调用，而是使用更高级的数据结构——例如开发者在 `safe Rust` 代码中执行堆分配时，其内部自动调用 `HeapAllocator`；当为跨域通信分配共享堆数据时，则调用 `SharedHeapAlloc`。开发者通过 `storage::insert/get/remove` 存储与检索状态数据，此类数据持久化存储于自定义堆中。

仅使用安全的 `Rust` 代码无法将所有的内核模块实现为安全的域，比如，一些模块需要处理内存读写或设备 I/O，这些操作是 `Rust` 编译器无法验证的，因此不可避免地需要引入 `unsafe` 代码来实现这些功能。然而，`unsafe` 代码是危险的，它不具备 `safe Rust` 的安全保障，如果不小心处理，很容易破坏域的隔离属性（`unsafe` 代码允许任意的内存访问）。为了对 `safe` 和 `unsafe` 的边界进行清晰的划分，Moat 在域基础库中对低级内核资源的不安全操作进行了封装和安全抽象，并为隔离域暴露出安全的接口。具体而言，域基础库中对内存读写、设备 I/O、指针操作、寄存器访问、同步原语等低级内核资源进行了安全抽象。

本文以 Linux 上的自旋锁安全封装为例，展示如何在 `Rust` 中封装一个更安全的自旋锁。在 C 代码中，开发者使用 `spin_lock_init` 函数初始化自旋锁，然后使用 `spin_lock` 和 `spin_unlock` 函数来加锁和解锁。这意味着开发者需要手动管理自旋锁的生命周期，并确保在使用完毕后正确释放它们，尽管直观来看这些操作是安全的，但在实际开发中，开发者可能会忘记释放自旋锁，从而导致死锁或其他问题，这在一

些复杂的函数中尤其容易发生。在 Rust 侧, 尽管可以通过 FFI 来调用这些接口, 但由于这些接口是 C 语言的, 因此需要将它们放置在 `unsafe` 代码块中, 从而在代码库中引入大量的 `unsafe` 代码。在 Rust 中, 最好的做法通常是将资源的管理权交给编译器, 由编译器来自动管理资源的生命周期。为了实现这一点, Moat 使用 Rust 定义了一个 `SpinLock` 类型, 并在其安全构造函数 `new` 中使用 `spin_lock_init` 初始化自旋锁, 并根据初始化结果返回一个 `Result<SpinLock, Error>`, `unsafe` 代码块被隐藏在 `new` 方法中。`Result<SpinLock, Error>` 类型比 C 语言的返回值要更符合语义, 因为 C 代码中仅仅使用负数表示错误值, 而这往往被调用者忽略, 相反, `Result<SpinLock, Error>` 类型强制调用者处理错误情况。`SpinLock` 拥有一个 `lock` 方法来加锁, 与 C 不同, `lock` 方法将会返回一个 `SpinLockGuard` 类型的对象, 这个对象实现了 `Drop trait`, 在作用域结束时会自动调用 `spin_unlock` 来释放自旋锁, 这是由 Rust 的所有权系统和生命周期管理机制自动处理的。这样, 开发者就不需要手动管理自旋锁的生命周期, 避免了忘记释放自旋锁导致的问题。

对于其它低级内核资源, 域基础库也提供了类似的安全抽象。通过这些抽象, 开发者可以在隔离域中安全地使用这些资源, 而不需要直接处理 `unsafe` 代码, 从而确保隔离域的安全性和可靠性。

3.3.2 基础库的安全抽象

尽管具体的实现会因为系统的差异而有所不同, 但 Moat 为基础库的设计和实现提供了一些通用的指导原则和类型抽象。

最小化原则 域基础库只会对最底层的低级内核资源和相关联的不安全操作进行安全抽象, 而可以由这些资源和安全接口组成的高级抽象和接口则是由域自行实施, 因为这些不再需要 `unsafe rust`。与 `redleaf`^[45] 中每个隔离域依赖不同的可信库相反, Moat 通过域基础库中的安全抽象为所有域提供了访问低级内核资源的统一途径, 避免了为大量外围设备驱动编写可信库的需要。而且, Moat 尽可能将更高级的抽象留到域中实现, 这在很大程度上减少了域基础库所包含的代码, 从而降低了对其内部 `unsafe` 代码的审查难度。

内核资源的安全回收 所有需要手动分配和释放的资源都必须进行封装并实现 Rust 内置的 `Drop` 特性。这将内核资源管理从开发人员的手动操作转变为编译器驱动的自动化操作。此外, 在发生域故障时, 故障处理流程也可以正确回收这些资源。通

过这种方式, Moat 在不破坏已有接口的情况下, 确保了隔离域中的资源使用是安全的。

域状态数据的安全存储 域使用域基础库中的安全抽象实现其功能, 这意味者一些内核资源可能构成域的状态数据。对于绝大多数资源来说, 它们是平凡的数据结构, 因此不会进行堆分配。然而, 少部分安全抽象内部可能会分配堆空间, 默认情况下, 这些堆分配发生在域的私有堆上。在域将资源作为状态数据来看待的情况下, 这些资源就需要被分配到自定义堆当中, 从而可以在域热更新后继续存在。为此, 域基础库为这些类型添加了一个额外的构造函数, 在这个额外的构造函数中堆分配被指定到域的自定义堆上。

内存访问的安全抽象 域基础库将域可以访问的内存分为两类, 类型化内存和无类型内存, 两者可以互相转化。域基础库定义了 `FrameTracker` 来表示无类型的内存, 其对应到一段连续的物理地址区间。在系统中, 无类型内存主要用于几个场景中:

- 用于域的细粒度堆分配;
- 用于与硬件交互, 读取硬件数据或写入数据到硬件中;
- 用于映射任务地址空间和访问用户数据。

类型化的内存包括域的代码、数据段、设备地址空间, Rust 堆。在域被加载后, 无类型内存自动转变为类型化内存, 并由 Rust 的类型系统接管。例外之一是设备内存, 无法从无类型内存转变到 Rust 中的对应类型, 域基础库引入一个 `SafeIORegion` 的类型来表示设备内存, 与其它类型的内存一样遵守 Rust 的所有权规则。在运行过程中, 域可以将 `FrameTracker` 动态地转换为具体的数据类型, 但是必须满足两个条件: (1) 无类型内存的地址需要符合数据类型的内存对齐规则; (2): 无类型内存大小不能小于数据类型的大小。

指针对象的安全抽象 在初始域集合被加载到内核后, 一些驱动域从域管理运行时获得设备信息, 并接管硬件设备。通常, 设备资源位于特定的内存地址区间, 因此对资源进行封装的数据结构需要一个指向其地址的指针, 在 `safe rust` 中, 域无法操作裸指针, 但却可以创建一个裸指针。为了防止域传递一个无效的裸指针, 域基础库引入了不可伪造对象, 它将裸指针作为一个 64 位大小的 `u64` 基本类型看待, 但被包装在 `SafePtr` 类型下, 这有点类似于类型别名, 但在 Rust 中 `SafePtr` 是一个与 `u64` 完

全不同的类型，SafePtr 的构造函数和大部分指针访问函数是 unsafe 的，这意味着域无法在 safe 的代码中随意构造和使用这个对象，但是域管理运行时可以构造，域基础库可以访问。在 Linux 系统中，不可伪造对象还被用于处理回调函数的重定向中 (§3.7)，因为这些回调函数的参数大多由裸指针表示。

3.4 隔离域管理运行时

表 3.2 域管理运行时的核心 API

接口	描述
DomainResource::insert/free_page_map	管理域分配的物理内存
storage::create_domain_storage	为域创建自定义状态存储堆
storage::remove_domain_storage	移除域的自定义状态存储堆
storage::move_domain_storage	更新域的自定义状态存储堆
DomainLoader::load/unload	加载域到内核地址空间/卸载域
DomainManager::register/query_domain	注册域/查询域

通常，不同系统的初始化代码是不一样的，与域基础库一样，Moat 也将域管理运行时分解为较小的组件，从而将域的加载器和管理器等基础设施与其它部分分离开。为了让 TCB 尽可能保持小巧，以此让绝大部分内核功能受益于语言的类型和内存安全，Moat 建议减少域管理运行时负责的功能，比如，域管理运行时响应硬件中断，但是处理中断的功能在对应驱动域中实现，域管理运行时进行任务切换，但是任务的生成和调度在不同的域中实现。

运行时提供用于管理物理内存、域存储和域生命周期的 API（表 3.2）。它包含分配和释放物理页、创建和移除域状态存储堆、加载和卸载域以及注册和查询域的函数。简单来说，运行时主要负责两部分功能：

- 硬件管理：它初始化硬件、管理物理内存、处理特权级切换以及处理硬件中断（与系统相关）。
- 域生命周期控制：它将域加载到内核地址空间或从内核地址空间卸载，从而实现隔离域的动态管理（与系统无关）。

系统初始化 运行时与动态加载的域不同，在系统启动时，运行时就已经被加载

到内存中运行，因此，直观上来看，运行时与传统的内核功能类似。在系统启动期间，运行时会建立初始页表，在探测可用的物理内存后，运行时需要再次建立完整的页表并对物理内存进行精细的管理。在完成基本所有的基本功能初始化后，运行时将会从预先指定的内存中读取并加载一些基本的域，这些域是整个系统实现正常运行所必须的。比如一个基本的 PCI 驱动域用于扫描 PCI 总线以识别设备，一个块设备驱动域用于读取磁盘，一个文件系统域用于访问磁盘上的文件，一个任务域用于创建进程。在建立第一个用户程序后，运行时就会切换到用户态并开始运行第一个用户程序。此后，运行时就陷入了等待中断或异常-进行中断异常的分发的循环当中。

域的管理 运行时还负责对域集合进行管理，在系统运行过程中，用户发出的域相关的请求由运行时处理，主要的请求包含三类：

1. 加载域到内核地址空间；
2. 从内核地址空间中卸载域；
3. 动态更新已加载的域；

在处理加载域请求时，运行时使用 `DomainLoader::load` 将域加载到内核地址空间，`DomainLoader::load` 实现了对域目标文件的解析以及重定位工作，在将所有需要加载的区域加载到内核地址空间后，运行时使用 `storage` 的 API 为域的状态存储创建自定义堆，共享堆是所有域共享的，因此在启动时已经创建。在完成这些基本工作后，运行时调用域的初始化接口，并在初始化接口中传递域自定义堆以及对共享堆的引用。尽管域的私有堆分配是在域基础库中完成的，但其底层使用运行时管理的物理页，这主要通过 `DomainResource` 接口来进行分配和释放，这些接口记录了分配元数据。自定义堆和共享堆底层同样也需要物理页分配，但不需要经过 `DomainResource` 接口，因为它们不需要记录元数据。

域准备就绪后，`DomainManager` 会将其注册到内核中，以供其他域查询和使用。比如，域可以使用 `query_domain` 接口获取其它域对象并调用其它域的功能，其参数是域的唯一名称。域的卸载是加载的逆过程，运行时需要回收其分配的资源。对于热更新，它的处理类似卸载，但与卸载不同的是，域的自定义堆被保留，因为状态数据需要迁移到新的域中，`storage::move_domain_storage` 用于更新域的状态存储堆。如图 3.1 所示，为了和已有的系统兼容，Moat 的运行时还需要协调原有的模块和域之间

3.5 域的隔离属性

图 3.2 Moat 的跨域通信

Moat 的隔离方法建立在 Rust 安全代码提供的四项基本安全保障之上，这些保障对于在软件中实现内存隔离至关重要^[88]。此外，Moat 依赖于三个关键组件：（1）Rust 编译器强制执行类型和内存安全不变量；（2）Rust 的 `core` 和 `alloc` 库（使用 `unsafe Rust` 实现基础类型）；（3）正确实现 TCB。值得注意的是，Moat 不解决硬件引发的内存安全问题（例如，内存管理单元 (MMU) 配置、中断控制器漏洞），假设这些组件在

这个信任模型中是非恶意的。

3.5.1 域间通信与隔离

Moat 通过将域的实现限定在 Rust 的安全子集之内并提供域基础库以封装域所需的 `unsafe` 操作来确保域获得了 Rust 语言提供的安全保证。然而，这还不足以实现完全的内存隔离，因为域仍然可能通过域接口传递包含复杂层次和引用关系的对象。为了解决这个问题，在之前的许多工作^[43,89]的基础上，Moat 引入私有堆和共享堆来满足域自身所需的堆分配和域间通信所需的数据传递。

私有堆 域的内部需要进行堆内存分配，Moat 通过为每个域建立一个私有堆来隔离域对堆内存的访问 (Figure 3.2)。与 C 语言的堆分配不同，Rust 的堆分配由一个全局的堆分配器进行管理，而每个被独立编译的域都需要声明其堆分配器，Moat 将这个分配器视作域的私有堆。域管理运行时管理域的功能之一是为域分配较大的页，并记录页的域所属权。在域内部，这些页被全局堆分配器用来分配较小的对象。采用两级分配器有助于域管理运行时在系统运行期间快速为域分配内存资源，并且在域进行卸载或更新期间可以安全且快速地回收这些资源。由于域只能从自己的私有堆上分配对象，域不能存在指向其它域的私有堆的指针。

域接口限制 通常，单个域可能并不能完成所有功能，需要通过域接口访问其它域的功能。为了阻止域接口成为私有堆对象的逃生通道，Moat 为所有域提供一个全局共享堆，并规定域间通信的数据共享通过共享堆完成。然而，单靠共享堆并不能约束域间传递数据的方式，比如，域可能将私有堆数据放置在共享堆 (Figure 3.2) 中进行传递。为此，Moat 引入了特殊的共享堆基本数据类型 `DBox<T>`，并在此基本类型上构建其它数据类型。`DBox<T>` 通过 Rust 的 `auto trait` 特性来实现 (1)：共享堆上分配的对象不能包含指向私有堆的指针；(2)：共享堆上分配的对象不能存储私有堆对象。此外，Moat 不允许域接口使用可变对象引用，虽然可变对象方便对数据修改，但会使得实施域的故障隔离变得困难。相反，Moat 使用不可变引用共享数据或通过移动共享堆对象所有权来实现对数据的修改。

`DBox<T>` 与 Rust 的内置类型 `Box<T>` 类似，但引入了一个额外的字段来记录其所属的域。在进行域间通信时，如果需要移动所有权，该字段会被更新为新的域标识符并在调用完成后恢复。移动所有权意味着调用者失去访问权。在域被卸载或更新时，域管理运行时扫描被分配的共享堆对象，回收所有权属于该域的共享堆对象。

直观来看，域间通信就是对域接口函数调用，这比那些需要进行特权级切换的方式要更高效。从本文实现的原型系统和在 linux 上的应用来看，对域接口施加的安全限制，并不会对域的功能开发产生额外的负担。

如何将已有的模块接口转换为域接口？尽管可以直接从零根据域接口定义实现一个域，但从简化工作的角度来看，将一个实现完成的模块改造为一个域是一个更好的选择。然而，由于隔离属性的限制，模块的接口不得不进行修改以满足其要求。这里以一个简单的块设备模块为例，展示如何将已有模块改造为一个域。如代码 1和2所示，改造前后的接口差异主要来自其参数上。在改造前，调用块设备驱动模块直接使用 `data: &mut [u8]` 读取磁盘数据，此时，这里的 `data` 所代表的内存仍然属于调用者，但是块设备驱动也拥有修改的权限，同样，使用 `data: &[u8]` 将数据写入磁盘也导致调用者的内存对块设备驱动模块可见，这和模块的内存隔离是相违背的，也就是说，作为被调用者，不应该拥有调用者的私有数据的访问权限，更不能拥有修改其私有数据的权限。在改造后，调用块设备驱动域接口的模块不再直接传递私有数据的可变引用或不可变引用，而是直接移动共享堆数据的所有权或使用共享堆数据的不可变引用。调用者在使用 `data: DVec<u8>` 读取磁盘时，`data` 的所有权被移动到块设备域中，此时其只能由块设备域访问，这是由 Rust 的类型系统保证的，只有块设备域正确读取了数据，才能将该数据的所有权返回给调用者。调用者仍然可以使用不可变的 `data: &DVec<u8>` 写入磁盘数据而不违背隔离规则，因为此时 `data` 并不是域的私有数据。

由于接口发生了改变，调用者也需要做相应的改变。具体而言，在调用块设备的接口前，调用者需要从共享堆分配对应的缓冲区，并将私有数据拷贝到该缓冲区中。在调用完成后，调用者可以将缓冲区的数据拷贝到内部的私有数据中。为了避免频繁的内存拷贝，调用者可以提前在内部分配共享堆的缓冲区，并直接在共享堆缓冲区上完成其功能。

```
pub trait BlkDeviceDriver: DeviceBase + Basic + DowncastSync {
    fn read_block(block: u32, data: &mut [u8]) -> Result<usize>;
    fn write_block(block: u32, data: &[u8]) -> Result<usize>;
}
```

Listing 1: 改造前的块设备接口

```
pub trait BlkDeviceDomain: DeviceBase + Basic + DowncastSync {  
    fn init(device: &Range<usize>) -> Result<()>;  
    fn read_block(block: u32, data: DVec<u8>) -> Result<DVec<u8>>;  
    fn write_block(block: u32, data: &DVec<u8>) -> Result<usize>;  
}
```

Listing 2: 改造后的块设备域接口

3.5.2 域故障隔离

鉴于操作系统的复杂性，消除所有错误是不现实的，但内核需要限制错误的影响范围，防止发生故障的模块破坏其它模块。在 Moat 中，内核分解和域的内存隔离属性促进了故障隔离的实施。分解意味着故障发生在较小的单个隔离域中，内存隔离属性确保了发生故障的域不会擅自修改其它域的内存。为了实现域的故障隔离，Moat 需要解决的一个核心问题是：如何正确处理故障？

Rust 语言使用 `panic` 来表达程序故障，根据编译时配置，程序可以选择在发生故障时直接终止程序的执行 (`abort`) 或者对执行流进行堆栈展开 (`unwind`)。为了捕获并处理域的故障，Moat 依靠 Rust 语言内置的 `panic` 钩子和故障捕获点。在编译时，Moat 会修改域的编译选项以启用 `unwind` 策略，域管理运行时在域加载时将编译器产生的 DWARF^[90] 信息一同加载。DWARF 是一种调试信息格式，提供了关于机器代码如何映射到源代码的信息，特别是用于跟踪栈帧的信息。DWARF 中有一种称为 Call Frame Information (CFI) 的结构，它记录了每个函数调用点的信息，用于帮助编译器和调试器知道如何展开栈。

在域内部，Moat 为每个域定义 `panic` 钩子，当域中出现故障时，执行流将会跳转到 `panic` 钩子中进入故障恢复流程。在钩子中，Moat 的展开器会沿着当前栈帧向上展开，即从当前的栈帧逐步退回到调用方的栈帧。在每一个栈帧中，它会依次调用栈中每个活跃对象的 `Drop` 方法，确保资源被释放。如果程序中有代码显式捕获了 `panic`，比如通过使用 `catch_unwind`，那么堆栈展开过程在捕获点停止，程序可以在捕获 `panic` 后执行特定的逻辑（如记录日志或恢复执行）。如果没有捕获到 `panic`，那么 `unwind` 过程最终会导致程序终止。为了将故障隔离在单个域中，Moat 将故障捕获点被定义在域的入口处，也就是在调用域接口的位置，这确保 `unwind` 最终始终会在故障捕获点处停下，结束故障处理。

使用 DWARF 进行堆栈展开时，展开器面对的是一个只有寄存器状态和当前指令

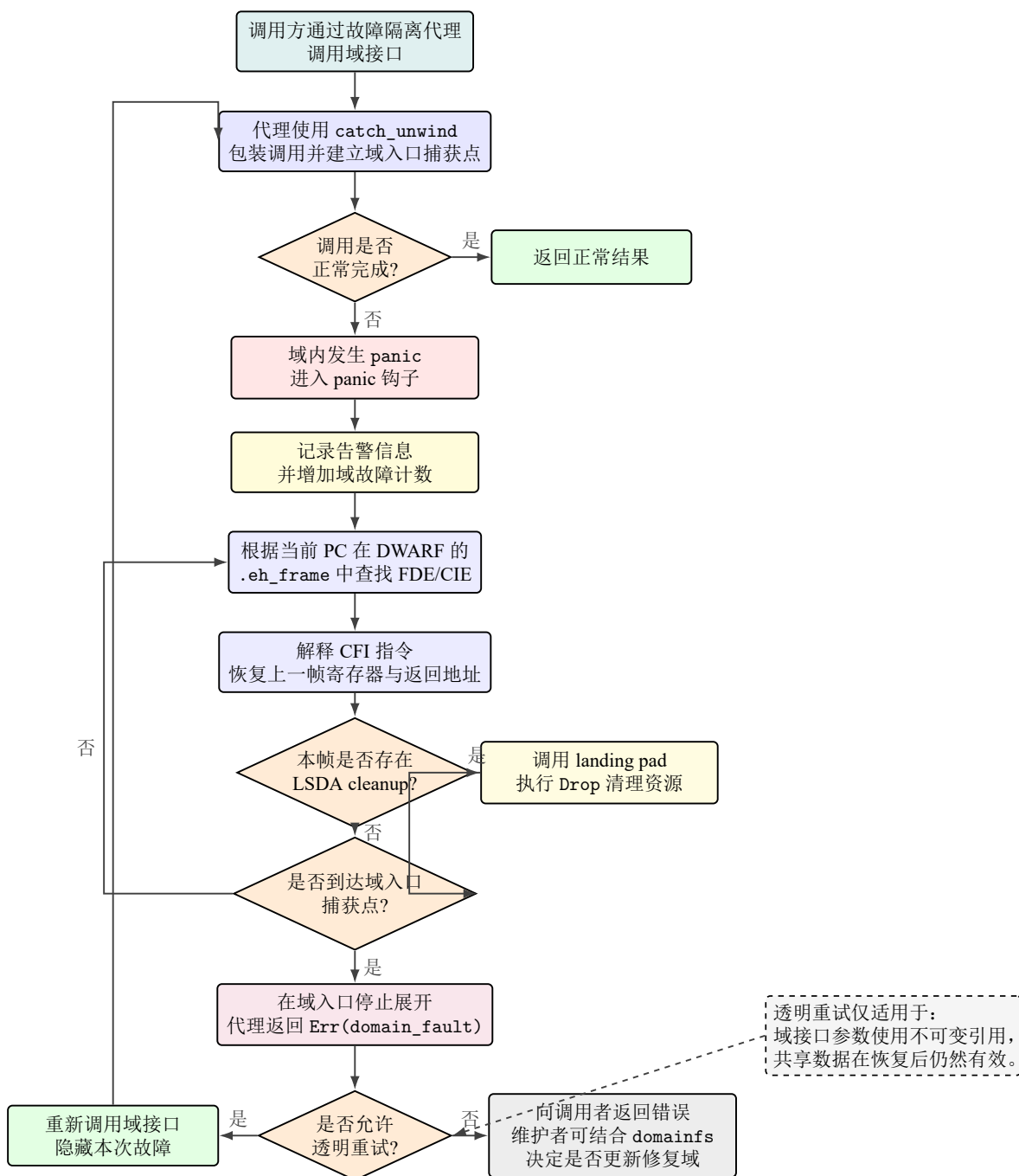


图 3.3 域的故障恢复流程

地址的机器现场，而 DWARF 中的 CFI 告诉它如何从这些初始状态一步步回溯，整个过程本质上是在解释 DWARF 的规则语言（CFI 指令）来恢复每一层调用帧的寄存器和返回地址。具体过程如下：在 panic 钩子中，展开器首先利用当前的 PC 地址去 DWARF 的 `.eh_frame` 中查找对应的 FDE（Frame Description Entry）。每个 FDE 都描述了某个函数的指令范围，从 PC 地址就能找到当前函数对应的那条 FDE。FDE 中包含两类信息：一部分是它自己的 CFI 指令，一部分是链接到该函数所属的 CIE（Common Information Entry），其中包含公共规则与基础约定。展开器先加载 CIE 和 FDE，然后执行它们所包含的 DWARF CFI 指令。这些指令不是机器码，而是描述性语言。展开器会按指令逐条解释，构建出一个虚拟的“帧描述状态”，这个状态告诉展开器：CFA（Canonical Frame Address）是什么、调用者的返回地址寄存器保存在什么位置、调用者的 SP/RBP/其他寄存器如何恢复。CFA 通常是“调用者栈帧顶部”的抽象地址，DWARF 会用它作为参考基准，计算出返回地址位于 CFA-8，保存的旧帧指针在 CFA-16。但编译器优化场景下，这些寄存器可能放在任意寄存器或栈槽里，或者只在某段指令范围内有效，这些变化都通过 CFI 的指令流描述出来。

当展开器解释完当前 FDE 的 CFI 指令后，它就得到了完整的“上一帧寄存器状态”。首先，根据返回地址规则，它会从 CFA+ 偏移处读取返回地址（RA），作为上一层函数的下一条指令地址。然后根据规则恢复上一帧的 SP、BP 和所有被调用者保存的寄存器。得到这些之后，展开器构造“上一帧的机器现场”：新 PC 地址、新栈指针和其他寄存器。接着，它将这一帧视作新的“当前帧”，再次根据返回地址去查找对应的 FDE，然后重复上面同样的解释过程。这就是堆栈展开的迭代方式：利用 RA 找到上一函数的 FDE → 执行 CFI 指令 → 还原上一帧寄存器 → 再往上。

此外，在恢复栈帧之前，展开器会查找当前函数的“异常处理表”（LSDA）。这个 LSDA 也是由编译器生成的，与 DWARF 结构紧密关联：其中的“action table”告诉展开器本帧是否包含需要在 panic 发生时调用的清理逻辑。Rust 语言就是通过这些 action entries 挂接自己的清理代码。如果该函数中有需要 drop 的变量，Rust 编译器会保证此处生成一个 landing pad，这个 landing pad 不是正常的函数控制路径，而是一个清理入口，用来执行生命周期中要求的 drop。在执行展开时，检测到当前帧有 cleanup action，就调用这个 landing pad。landing pad 内部再调用 Rust 的 drop 胶水代码：这是编译器为特定类型自动生成的释放逻辑，例如 Drop trait。

通过为内核资源实现 Drop 特性，Moat 在堆栈展开期间自动释放这些资源，从而

防止故障处理后出现资源泄漏和死锁。这种异常处理方法在 C 语言中更具挑战性,这也使得 Moat 可以简化对不同资源的处理,从而避免像 SafeDriver^[10]使用复杂的资源操作记录来处理不同类型的资源。此外,得益于域的隔离边界,故障恢复很快在域入口处结束而不需要对任务执行流执行完整的堆栈回溯^[44,91],这减少了故障恢复花费的时间。

执行流的堆栈展开不会影响域的服务,其它位于域中的任务的执行流仍然正常进行。但由于域出现了故障,有很大概率是域的实现出现了错误,为了表达这一特征,在 panic 处理流程中,Moat 发出警告信息并记录域发生崩溃的次数。在用户态,Moat 暴露了一个 domainfs,向系统维护者展示系统中所有域信息,同时也包括域的故障计数。维护者可以根据故障计数判断错误的严重程度并决定是否对域进行更新修复。

不过,基于 unwind 的故障处理不是万能的,首先,Rust 的 panic 不能发现所有的故障,比如,堆栈溢出将会被 MMU 发现并被操作系统结束任务,硬件错误也不会被 panic 所发现。另一方面,当故障发生在域修改了一些内部状态之后,故障恢复将不能撤销这些修改,这可能会导致域出现更多的故障,此外,故障恢复期间如果系统内存不足将会导致故障恢复失败。这些问题已经超出了故障隔离的能力范围,因此,Moat 将责任推送给系统维护者。

故障隔离代理 Moat 依靠故障隔离代理来自动实施故障隔离从而向外部隐藏处理细节。具体而言,故障隔离代理将真正的域对象进行封装,并且实现和域对象相同的 trait,这相当于对域接口调用进行了一次中转。在内部,故障隔离代理将域对象的所有接口调用包装在如代码 3 所示的 catch_unwind 函数中,这个函数使用 Rust 内置的特殊方法使得编译器生成故障捕获点,也就是生成了 DWARF 信息。相比在接口调用时保存任务寄存器信息,这种方法不会给域间通信带来较大的开销。在域正常工作期间,这个函数包装不会对域接口调用带来额外的副作用,只有在域发生故障时,故障恢复流程最终将暂停在该捕获点处并返回一个错误以表征域内发生了故障,这样,发生故障的域就不会向上返回错误的数据。这对一些设备驱动域是有用的,这些设备可能因为某些原因导致请求失败而需要重试。

```
fn catch_unwind<F: FnOnce() -> R, R>(f: F) -> KernelResult<R>
```

Listing 3: 故障捕获接口

Moat 进一步扩展了故障隔离代理的功能，如果内部对象的调用返回了 `Err(_)`，除了直接向上返回错误外，它还允许对特定的调用进行重新调用从而向调用者隐藏域故障情况。但重新调用并不是免费的，在故障恢复进行堆栈展开期间，由域接口传递到域中的共享堆数据会被释放，所以故障隔离代理无法使用相同的共享堆数据进行重新调用。一种可能的方法是在第一次发出调用时拷贝一份共享堆数据，但这种做法开销过大。因此，故障隔离代理只允许在域接口使用不可变引用共享数据的情况下进行重新调用，Rust 编译器确保了不可变引用的数据不会被修改，而引用不会被释放。

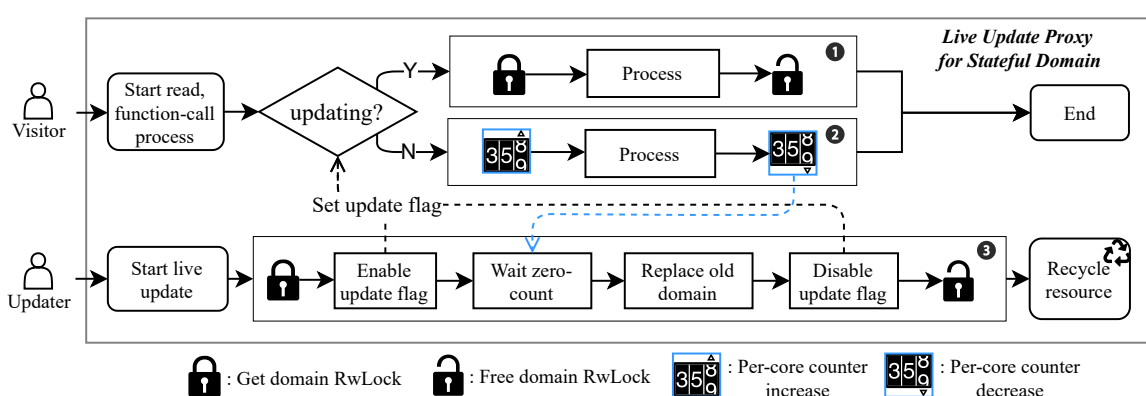


图 3.4 有状态域热更新过程

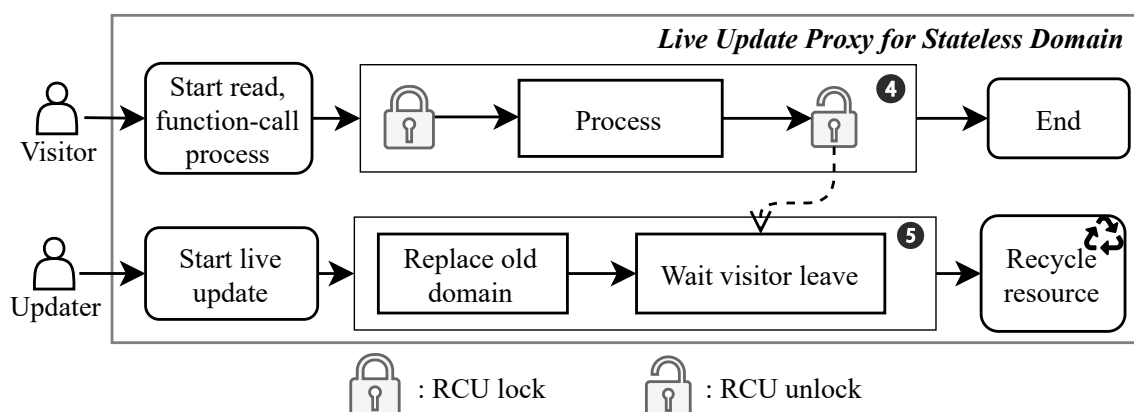


图 3.5 无状态域热更新过程

3.6 隔离域的热更新

在像 Linux 这样的内核中做动态升级是困难的，最重要的原因是内核状态数据的复杂性，内核模块之间互相交织导致一个模块的状态可能被多个模块共享和修改。基于语言机制的模块隔离促进了域的状态数据隔离，域之间通过接口访问功能，因此域不会共享特殊的全局状态，每个域是独立的部分，它们的状态数据也是独立的。已有

的工作大多将隔离和热更新视作两个独立的问题进行解决，或者像 Theseus^[44]另辟蹊径以无状态模块设计解决热更新的问题。与这些工作不同，Moat 将热更新和隔离结合起来一起考虑，这不仅可以大大简化热更新的实施，还不需要推翻现有的常见的有状态模块实现方法。

3.6.1 状态保存和恢复

热更新的核心在于确保域在升级前后拥有相同的正确的状态。当深入地考察域更新前后所发生的变化，本文发现，对一些域的更新实际上不需要状态数据的保存和恢复，Moat 把这种域称为无状态域。这里的无状态指的是域内部的数据是无关紧要的，不需要在热更新的时候考虑。对于其它的域，Moat 称为有状态域，有状态包含两层含义：（1）这些域内部确实保存了需要在更新前后保持一致的状态数据，比如任务管理、虚拟文件系统等；（2）这些域更新后可以从一个初始状态重新开始运行，但是这些域可能会独占机器的硬件设备，比如底层设备驱动程序。

通过对有状态域的分析发现，域中分配的大多数数据都会很快被释放掉，但是对于域运行至关重要的状态数据则通常不会马上释放，并且很多时候都被保存在堆内存或者全局变量中，这些状态数据仅仅占据域数据的一小部分。Plugsched^[15]正是利用了这一点，结合模块中特定 API，它将模块中的状态数据以类似“pop”的操作移出旧的模块，然后以类似“push”的操作移入新的模块。然而，这种方法过于和模块 API 绑定，通用性比较差。Moat 从另一个角度出发：

1. 既然模块的状态数据和普通数据不同，那么就直接在定义时区分两者的存储方式；
2. 既然模块状态数据需要在模块的整个生命周期中存在，那么只需要将状态数据保存在一个可以持续存在的地方即可。

```
fn insert<T>(key: &str, value: T) -> Option<Arc<T, CustomStorage>>;
fn get<T>(key: &str) -> Option<Arc<T, CustomStorage>>;
fn get_or_insert<T,F>(key: &str,f: F) -> Arc<T, CustomStorage>;
fn remove<T>(key: &str) -> Option<Arc<T, CustomStorage>>;
```

Listing 4: 自定义堆接口

自定义堆 Moat 利用 Rust 的自定义堆分配器实现状态数据的分离存储。自定义

堆分配器允许将分配的对象存储在一块特定的堆内存中。为此，Moat 为每个域引入了一个自定义堆分配器。如代码 4 所示，域基础库向域提供了分配、删除和索引接口，分配接口用于域进行对象分配，索引接口用于域在升级后索引已经保存的状态数据。在升级期间，域的资源被域管理运行时回收，包括其使用的私有堆，代码段、数据段等，而在自定义堆中保存的状态数据则被保留下来，自定义堆的域所属权会被更新为新的域，更新之后的域将直接继承这些数据。

```
type __TaskList = Mutex<VecDeque<DBox<TaskSchedulingInfo>, CustomStorge>>;
type TaskList = Arc<__TaskList, CustomStorge>;
pub struct CustomFiFoScheduler {
    tasks: TaskList,
}
impl CustomFiFoScheduler {
    pub fn new() -> Self {
        let task_list = get_or_insert::<__TaskList, _>("tasks", || {
            __TaskList::new(VecDeque::new_in(CustomStorge))
        });
        Self { tasks: task_list }
    }
}
```

Listing 5: 域状态数据的存储示例

自定义堆的引入带来了对象分配方式上的改变，Moat 尽量保持接口简洁易用以降低开发负担。为了让隔离域支持热更新，域开发者需要

1. 区分域内的状态数据和非状态数据；
2. 使用域基础库提供的分配、删除和索引接口进行状态数据的管理。

代码 5 所示展示了一个简单的任务调度器域如何保存其状态数据。首先，域需要定义其状态数据类型，这通常是指定数据需要存储在 CustomStorge 自定义堆上，然后在域的初始化过程中，域可以使用 get_or_insert 接口获取或者分配状态数据，通常，域的第一次运行会分配数据，而当域被更新时，新域则会直接获取到旧域的数据。这种方式在通用性和灵活性上取得了折衷，开发者需要对代码做一些修改，但对所有的域都适用。不过，受限于是这是一种语言提供的机制，还暂时不能将其应用到已有的使用 C 实现的模块中。

3.6.2 状态同步

针对有状态和无状态域，Moat 应用不同的状态同步机制。在升级内核模块时，较大的服务暂停时间会影响系统提供的服务。最理想的情况是在更新时保持服务而不中断，而对无状态域的划分可以实现这一目标。域是无状态的，意味着域对外提供服务并不依赖自身存储的数据，这意味着在进行无状态域升级时进行以下操作：

1. 旧域仍然向外提供服务，并不会在更新时立即从系统中移除；
2. 新域被加载到内核中，新的任务使用新域的服务；
3. 在旧域中活动的任务全部离开后，域管理运行时再卸载旧域，此后所有的服务由新域提供。

无状态域的 RCU RCU^[92]是一种同步机制。它允许读者在不加锁的情况下读取共享数据，同时确保写者能够安全地更新数据，而不会影响正在进行的读操作。Moat 使用 Linux 内核中流行的 RCU 机制来完成无状态域的升级。简单来说，Moat 把无状态域视为需要并发访问的数据，域的普通访问者作为读者 (图 3.5 路径 ⑤)，域的更新者作为写者 (图 3.5 路径 ④)。这样就可以把 RCU 的优势完美应用到对无状态域的热更新中。当新的域加载到内核中时，旧的读者可以继续使用旧域提供的功能，而新的读者不用等待所有旧读者离开就可以直接使用新域提供的功能。域的更新者会使用 RCU 同步原语确保所有读者离开旧域，域管理运行时就可以安全删除旧域所有的资源。

与 Theseus 的无状态模块不同，其无状态模块对外表征并非一个独立的对象，因此在更新期间需要重写模块之间的依赖关系。在 Moat 中，访问域对象是通过指针间接进行的，从而可以实现更新对象时的原子性，因此可以直接使用 RCU 机制来避免服务暂停。

有状态域 有状态域与无状态完全不同，其提供的服务依赖内部的状态数据，如果在更新期间有其它任务位于旧域中，并在更新域的同时修改了域中的状态数据，就会造成状态数据不一致，引发严重的错误。因此，有状态域的更新需要确保所有的任务已经离开旧域并阻止新的任务进入旧域中才能安全地重新初始化新域。

为了检查所有任务是否位于模块中并进行升级，大多数补丁工具 (比如 kpatch^[11]) 使用 Linux 的 `stop_machine` 接口来让 CPU 进入静止状态，在状态静止期间，特定的 CPU 执行用户定义的处理程序，其余的 CPU 停止工作。Plugsched^[15]进一步使用任务

并行和二分搜索来加速任务检查。但是 `stop_machine` 仍然造成了比较长的服务暂停时间，它还会让正在执行其它任务的所有 CPU 停止，即使这些任务跟需要升级的模块并不相关。造成这个问题的原因是现有的模块缺乏一个明确的边界，无法直接统计当前位于模块中的任务。

动态使能锁 隔离的一个关键属性允许 Moat 执行快速的任務检查然后进行升级：域接口构成了一个明确的边界。Moat 引入了读写锁来进行同步，从而确保在更新期间没有任何任务位于旧域中。具体来说，不管是普通的访问者还是更新者，都会先获取锁再调用域接口。然而，当大量任务调用域接口进入域中，在每一个接口上都用锁带来的性能开销是巨大的。为了解决这个问题，Moat 采用一种折衷的方法，具体而言，Moat 为域增加了一个全局标记，在有状态域的正常運行期间，所有的域调用将会走一条没有锁的代码路径 (图 3.4 路径②)，此路径使用一个 `per-CPU` 变量对任务的进入和离开进行计数。当有更新者进行域更新时，更新任务将提前获取锁，并使能全局标记 (图 3.4 路径③)，后续的访问者因为全局标记被使能而走有锁的路径 (图 3.4 路径①)，但因为更新者已经获得锁，它们需要等待升级结束才能获得锁。

1	Visitor1(core1)	Updater(core2)	Visitor2(core3)
2	check flag	lock()	
3		enable flag	
4	---IRQ---		check flag
5		cal counter	lock()
6	inc counter	update domain	

Listing 6: 并发错误

1	Visitor1(core1)	Updater(core2)	Visitor2(core3)
2	disable IRQ		
3	check flag	lock()	
4		enable flag	disable IRQ
5			check flag
6	inc counter	send IPI	enable IRQ
7	enable IRQ	cal counter	lock()

Listing 7: 并发错误修复方法

竞争条件 在更新者使能标记后，它统计所有 CPU 上的任务计数器从而确保所有访问者已经全部离开。如果计数器之和为 0，意味着进入安全状态，可以对域进行更新。在这里，动态使能锁在多核情况下引入了一个微妙的竞争条件：如代码 6 所示，

cpu2 的更新者使能标记前一刻, cpu1 的访问者 1 已经读取了标记, 其会走无锁路径, 但是在更新者统计任务计数前, 访问者 1 的执行被中断打断 (第 4 行), 由于访问者 1 没有更新任务计数器, 更新者会误认为可以进入安全状态, 中断结束后访问者 1 才更新计数器并且进入到了旧域中。其它在更新者使能标记后读取标记的访问者不会受此影响, 因为即使中断发生在检查标记和获取锁之间, 中断结束后, 它们依然会被锁阻挡。

Moat 使用一个巧妙的方法解决这个问题, 如代码 7 所示, 首先, 在访问者检查标记前, 关闭当前 CPU 的中断, 如果标记指示走无锁路径, 那就在完成计数器增加操作后, 重新开启中断, 这样这段代码就不会被打断, 如果是有锁路径, 那么在检查标记后就可以重新开启中断。然后, 对于更新者, 在统计任务计数前, 它需要向其它 CPU 核心发送一次核间中断, 并等待其它 CPU 响应 (第 6 行)。当其它所有 CPU 都响应了这个核间中断 (cpu1 的第 7 行后), 意味着其它 CPU 上走无锁路径的访问者已经全部完成了计数器更新, 此时更新者才能正确地统计任务计数。域管理运行时调用新的域对象的初始化接口完成新域的初始化, 然后将新的域对象替换掉旧的域对象从而完成升级。由于域的状态数据存储自定义堆中, 只需要将该旧域的自定义堆的标识符更新为新域即可完成状态数据的迁移。

通过采用优化的读写锁, 热更新不会在有状态域的正常执行过程中引入额外开销, 同时避免了在更新期间使用代价高昂的 `stop_machine()` 接口进行任务堆栈检查。这显著减少了可能导致的服务停机时间。所有这些优化都源于最基本、最关键的属性——隔离。

热更新代理 与故障隔离代理类似, 热更新代理的目标是向外部隐藏具体的热更新处理逻辑。热更新代理会将故障代理进行封装, 并且也会实现和域对象相同的 `trait`。对于域开发者来说, 它们使用其它域的方式是接口对象, 因此, 热更新代理和故障代理对域开发者来说是透明的。域间通信在热更新代理中进行第一次中转。热更新代理的主要功能由两部分组成, 首先, 针对无状态域和有状态域, 热更新代理将内部对象封装到不同的同步数据结构中, 对于无状态域, 热更新代理使用 RCU 数据结构, 对于有状态域, 热更新代理使用 per-cpu 计数器和读写锁。其次, 热更新代理在进行热更新期间进行对象的替换和状态数据同步。

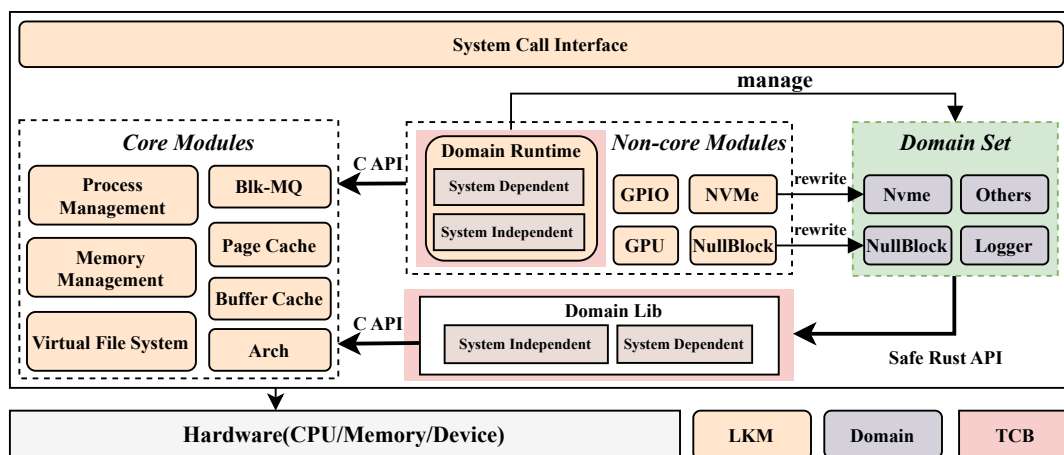


图 3.6 Moat 适配方法

3.7 Moat 适配方法

这里，本文以 Linux 操作系统适配 Moat 架构为例。图 3.6 展示了 Moat 如何集成到 Linux 内核中。Moat 不会破坏现有的 Linux 内核结构，而是添加一个新的内核模块作为域管理运行时来为隔离模块提供服务，而域基础库则作为安全的抽象层，使用 Rust 的安全特性封装现有的内核资源抽象。在这种情况下，域管理运行时和域基础库子模块中与系统无关的部分可以直接在 Linux 内核中重用。然后，开发人员可以使用安全的 Rust 重写现有的内核模块，将其转换为隔离域或创建新的隔离域。这种集成方法使 Moat 能够利用现有的内核结构，同时为内核模块引入健壮的隔离机制。

域基础库的移植 本文从系统无关和系统相关的部分来分析域基础库。作为一个库，域基础库无需作为单独的 LKM 存在，相反，它被作为依赖库编译到隔离域的可执行文件中。由于域基础库由一系列小型库组成，因此系统无关部分无需修改即可重用，而系统相关部分可以利用现有的内核基础架构。例如，Linux 内核已经为内核资源提供了足够的抽象，无需从头构建这些抽象。然而，由于 C 语言实现与 Rust 的类型系统之间存在显著差异，需要使用 Rust 封装现有的 C 语言接口和数据结构。因此，内核资源的安全抽象变成了内核模块导出的外部接口的封装（图 3.6），还可以直接使用 Rust for Linux (RFL) 项目^[84]中已经实现的封装来简化这部分工作。

域管理运行时的移植 与域基础库不同，域管理运行时模块是一个需要随内核运行的组件。因此，在现有系统中实现域管理运行时需要在内核源代码中添加代码。然

而，在 Linux 中实现域管理运行时，可以将其作为一个新的内核模块添加，而无需修改内核源代码（图 3.6）。与域基础库类似，域管理运行时中与系统无关的部分可以直接重用。而与系统相关的部分则需要根据现有内核结构进行调整。在 Linux 中，域管理运行时中与系统相关的部分无需重新实现，而是直接使用 Linux 内核中其他模块提供的 API。例如，为了使用 DomainLoader 实际加载域，Moat 可以使用 Linux LKM 的模块加载接口来分配虚拟地址空间。

这种分解有两个关键优势。首先，它将隔离覆盖范围扩展到大多数内核模块。其次，将隔离基础设施与模块实现解耦，可以实现不同系统的无缝重用——这对于实际应用至关重要。

3.8 本章小节

本章系统阐述了 Moat 内核模块隔离框架的设计方案。该框架的核心在于构建一个基于 Rust 语言安全特性的可信计算基，其由隔离域基础库和域管理运行时组成，负责安全地封装底层操作并提供隔离机制。内核功能则被实现为完全由安全 Rust 编写的隔离域，通过明确定义的接口进行交互，从而实现了隔离机制与业务逻辑的解耦。

Moat 通过为每个域建立私有堆来实现内存隔离，并通过全局共享堆及严格的类型限制来安全地实现域间通信。域基础库将不安全的内核资源操作封装为安全抽象，而域管理运行时则负责域的加载、卸载和生命周期管理。基于此隔离架构，Moat 能够利用 Rust 的异常处理机制实现故障隔离，将模块故障限制在域内。

框架的创新性尤为体现在将隔离属性作为实现高效热更新的基础。通过引入自定义堆来显式管理域状态，Moat 能够支持域的热更新。针对无状态域，采用 RCU 机制实现服务不中断的更新；对于有状态域，则利用隔离边界实现快速任务检查和状态迁移，显著减少了服务暂停时间。

本文还以 Linux 为例介绍了适配 Moat 的方法，得益于良好的分解，Moat 可以方便地插入到现有系统中，这避免了对现有系统进行破坏性的更改，又可以逐步引入隔离的机制。总之，本章详细描绘了 Moat 框架如何通过语言安全、架构解耦和精巧的机制设计，系统地解决内核模块隔离、故障隔离和热更新的难题，为构建更安全、更可靠、更易维护的操作系统内核提供了一个切实可行的设计方案

4 Moat 的实现

```
#[proxy(BlkDomainProxy,RwLock,Range<usize>)]
pub trait BlkDeviceDomain: DeviceBase + Basic + DowncastSync {
    fn init(device: &Range<usize>) -> Result<()>;
    fn read_block(block: u32, data: DVec<u8>) -> Result<DVec<u8>>;
    fn write_block(block: u32, data: &DVec<u8>) -> Result<usize>;
}
// export macro
gen_for_BlkDeviceDomain!();
define_unwind_for_BlkDeviceDomain!(BlkDomain);
```

Listing 8: Rust 宏标注

为了验证 Moat 的实用性，本文首先在 riscv64 平台上从零实现了一个原型系统 Alien OS，除了部分与硬件相关的低级汇编代码，超过 98% 的 AlienOS 代码完全由 Rust 进行编写，为了进一步验证 Moat 的通用性，本文将 Moat 应用到 Linux 系统中，并将已有的 Rust 内核模块改造为隔离域。

本文为 Moat 实现了用于支持模块隔离和热更新的基础设施，它包含 3.5k 行 Rust 代码。目前，这些基础设施作为一系列小组件存在，在不同的系统中，它们将分别被 Moat 中定义的运行时和域基础库所使用。除第三章描述的这些关键基础设施外，本文还实现一些额外的功能和工具来减少开发人员的工作。本章将详细描述这些额外的功能和工具，并介绍原型系统 Alien OS 实现和 Linux 适配的情况。

4.1 辅助工具

代码生成 本文观察到，对于所有的域来说，他们拥有相同的域基础库初始化代码，域的更新代理对象和故障代理对象与域对象实现相同的域接口，同类型的域会实现同一个域接口。解决代码重复实现的简单方法是引入一个 IDL 语言和编译器，编译器根据接口定义生成代码，这也是大多数类似工作的选择，但这会导致额外的学习成本，并且可能无法高效地和系统进行同步。Moat 摒弃了这种做法，相反，Moat 完全使用 Rust 来生成隔离域的模板代码，并使用 Rust 的宏来完成域代理对象的接口实现。如代码8所示，在定义域接口时，开发者可以给接口添加一个 Proxy 属性，并指定域代理名称、更新代理对象使用的同步方法，以及域初始化参数类型。根据这些参数，Moat 定义两类域代理对象的具体实现，并导出两个宏接口。使用这两个宏接口，

Rust 编译器自动生成预定义代码。

用户工具 为了简化开发者创建、实现和编译域的流程。Moat 提供了自动化工具，开发者以交互式的方式选择不同类型的域，并提供域要实现的域接口，工具将会为用户创建一个 Rust 项目，包含一个库程序和一个可执行程序以及它们的基本依赖。开发者像开发库一样提供域的实现，不同的是，库不向外暴露接口，开发者在预定义的虚拟 `main()` 函数中返回实现了域接口的域对象。可执行程序并不需要开发者编辑，它会自动引用库程序。稍后，用户可以选择编译指定的域，自动化工具将会自动搜索域位置并编译其为位置无关可执行程序。

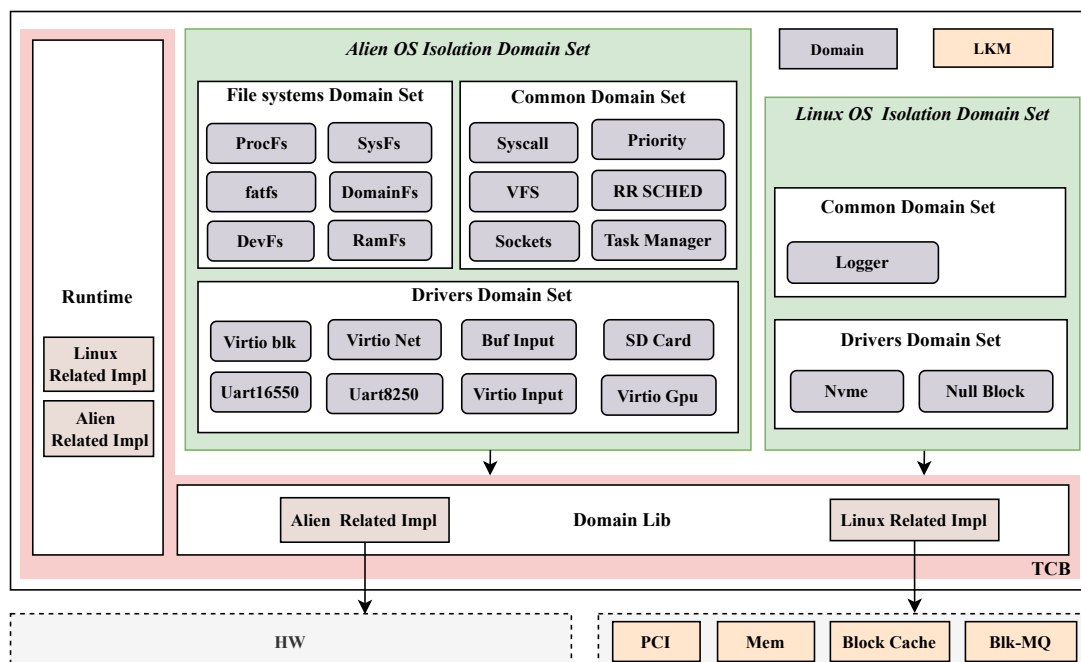


图 4.1 Moat 在 AlienOS 与 Linux 中的区别

4.2 Alien OS

AlienOS 大约包含 30k 行 Rust 和 200 行左右的汇编代码，拥有完整的文件系统和网络堆栈。其中，TCB 包含 7k 行代码 (3k 行是域的基础设施代码)，完全由 safe rust 实现的域集合包含 23k 行代码，占 74%。如图4.1所示，Moat 的三个核心部分自底向上构成了 Alien OS 的完整形态。其中，TCB 中的运行时负责初始化硬件并加载各个隔离域，除此之外的大部分内核功能全部被划分为独立的域进行实现，包括任务管理、调度器等核心模块。Alien OS 划分了几个不同的集合：通用域集合、文件系统域集合、

驱动域集合，每个集合中都包含了多个域，它们都依赖域基础库提供的安全抽象。许多域都可以在运行中实现热更新，比如调度器。从零开发允许本文完全按照 Moat 的倡议和方法使用 Rust 实现一个功能完整的操作系统，从而验证基于语言机制实现模块隔离和热更新的可行性。

4.3 Linux 内核模块

在 Linux 中使用 Rust 进行编程依然处在早期尝试阶段，本文的工作建立在 RFL^[84]项目上。本文使用 Rust 实现了 2k 行的内核模块作为 Moat 的运行时，RFL 数据结构的安全抽象被迁移到域基础库中，并根据架构设计增加了大约 1k 行代码用于封装内核 API 和添加热更新所需支持。此外，本文还分别修改了 `rmull`(150 行)，`rmvme`(300 行) 两个使用 Rust 实现的 lkm，以将其改造为隔离域。

与从零开发的 AlienOS 不同，Linux kernel 已经是一个功能完善的复杂系统，因此需要将 Moat 作为 Linux 的扩展进行实施。值得一提的是，Linux 和 AlienOS 共享相同的隔离和热更新所需的基础设施代码(图 4.1) 以及自动化工具。鉴于两个操作系统巨大的差别，在运行时和域基础库中，需要修改了系统相关的实现，比如，运行时中，可以使用现有内核模块提供的功能为域的加载分配虚拟地址空间，在域基础库的安全抽象中，不再需要像 Alien OS 一样直接处理硬件设备的不安全操作，因为这些不安全操作在 Linux 的现有内核模块中已经经过了抽象，因此，可以直接对现有的 C 接口和数据结构进行封装和安全抽象。尽管当前在 linux 中 TCB 的全部代码数量(10k, 包括 3.5k 的域基础设施代码)) 是域代码数量(2.7k) 的三倍，但随着域数量的增多，更多的代码将会由 safe Rust 构成。

内核回调函数重定向 在 Linux 中，内核可加载模块的常用做法是通过向内核注入回调函数来扩展内核的功能。模块可以使用内核导出的符号，向内核申请资源或者释放资源。与 LKM 不同，域不能通过注册回调函数来扩展 kernel 的功能，因为 Moat 要求对域功能的访问全部从域接口进入。

为了让所有对域功能的访问通过域接口，需要禁止域注册回调函数。可以通过创建中介函数来把内核回调函数重定向到域接口上来解决这个问题，这基于对内核回调函数接口的观察结果上：这些回调函数参数指定的数据结构中，通常存在一个字段用于保存实现者的私有数据。具体而言，Moat 按照内核回调函数接口定义来定义域的接口，以尽可能减小两者的差异，在域初始化内核数据结构期间，域对象会被保存在

私有数据字段中，在内核调用回调函数时，首先调用定义的中介函数，在中介函数中，从私有字段中取出域对象，并调用域的接口，当这些数据结构被销毁时，从中释放域对象。

内核编译 本文还修改了 Linux 内核的编译配置，实验中观察到在故障恢复期间以及驱动初始化设备期间，内核发生了崩溃，其原因是任务内核栈发生了溢出。通常，Linux kernel 为每个任务分配大小为 8k 的内核栈，这在绝大部分情况下是合理的。然而，Rust 编译器尝试在栈上为堆变量分配空间，并在稍后拷贝到堆上，这导致内核栈被快速地消耗掉。本文增大了内核分配给任务的内核栈大小来解决这个问题。当前 Rust 社区正在对编译器进行优化以减小 Rust 程序对栈空间的不必要消耗。

4.4 本章小结

本章对 Moat 的实现进一步做了补充和描述，除了第三章设计方案中描述的内容，Moat 还实现了一系列辅助工具来简化用户开发。首先，隔离代理，热更新代理等通常实现域对象相同的接口，并且使用相同的内部逻辑，为了降低用户重复粘贴代码的负担，Moat 利用 Rust 宏来自动生成模板代码。其次，域的开发有一套标准的流程，Moat 提供了一个自动化工具，提供了交互式的接口让用户快速创建一个域模块的基本代码结构，并处理其编译过程。

本章还介绍了 Moat 的两个案例，一个是完全从零实现的 AlienOS，一个是将 Moat 移植到 Linux 内核中并对 Linux 已有模块进行域改造。从零开发允许完全按照 Moat 的倡议和方法使用 Rust 实现一个功能完整的操作系统，从而验证基于语言机制实现模块隔离和热更新的可行性。而 Linux 内核模块的改造则展示了 Moat 设计的通用性以及适配 Moat 可能需要的更改。

5 实验评估

在评估时，本文想要回答以下问题：

1. 为支持模块隔离和热更新，Moat 的域抽象带来的开销有多大？ (§5.2)
2. 故障恢复如何影响应用程序以及需要付出什么代价？ (§5.3)
3. Moat 的热更新如何发挥作用，热更新的服务暂停时间有多大？ (§5.4)

5.1 实验环境

对于 Linux 驱动的隔离域改造，所有基准测试均在一台配备了 Intel i9-13900HX 处理器、32GB RAM 的服务器上运行，服务器运行 64 位 Ubuntu22.04，内核版本为经过简单修改 (第 5.2 节) 的 linux 6.8.0，该服务器配备了一块 1TB Samsung SSD 990 PRO。在运行测试期间，所有 CPU 核心运行在 performance 模式，并且禁用了超线程，TurboBoost 以及 CPU 频率缩放。测试所用的核心被固定在 2.1GHz 以减少性能测试中的差异。在对 NVMe SSD 进行测试时，SSD 会被提前进行格式化，以便每次测试时它都处于可比较的状态。此外，本文展示了在 AlienOS 原型中对任务调度程序进行实时更新的可行性，并在配备四核 1.5GHz 处理器和 8GB RAM 的 RISC-V 开发板 VisionFive2^[93]上进行了评估。

5.2 隔离开销

5.2.1 跨域调用开销

为了更详细地了解 Moat 与同样利用语言机制进行隔离的方案 (RedLeaf^[45]) 的区别。本文进行了一个微基准测试，以测量在进行域间调用时携带不同数量参数和不同配置下的开销。所实现的测试接口只是简单地对传入的参数进行自增并返回结果。表 5.1 显示了以 CPU 周期为单位的调用域接口的成本，包括无参调用以及单个共享对象传递调用。其中，RedLeaf 允许使用 shadow 机制以提高隔离模块的可靠性，Moat 默认支持隔离模块的热更新，也允许移除热更新功能。为了便于参考，本文还统计了简单系统调用 (getpid) 开销和使用最新隔离机制 MPK 的指令执行开销。

测试结果表明，在全功能状态下，与同样利用语言机制进行隔离的 RedLeaf 相比，

表 5.1 跨域调用的时钟周期数

Operation	Cycles
RedLeaf (no arguments)	113
RedLeaf (passing an RRef<T>)	134
RedLeaf via shadow (no arguments)	290
RedLeaf via shadow (passing an RRef<T>)	326
Moat (no arguments)	50
Moat (passing an DBox<T>)	99
Moat (no arguments, no live update proxy)	5
Moat (passing an DBox<T>, no live update proxy)	49
MPK (single instruction)	40
System call (getpid)	270

Moat 可以将域间调用的开销降低到 100 个周期内。这些改进主要来自两个方面，首先，Moat 不需要通过保存任务状态来实现域的故障隔离，而任务状态的保存和恢复会在每次域间调用上带来 80-90 个周期的开销。此外，Moat 不用引入 shadow domain 来重启故障域，因为 Moat 可以通过语言级机制处理域故障并通过热更新来修复域错误。本文在实验中还观察到，Moat 的域间调用开销中很大部分来自对热更新的支持，在移除热更新代理之后，域间调用下降到 50 个周期内。

即使是最简单的系统调用 getpid，也要花费大概 270 个周期，相较而言，利用语言机制的方法大大减少了上下文切换带来的开销。Intel CPU 引入的新的隔离硬件 MPK 可以用来实现高效的进程内隔离，单个 MPK 指令花费大约 40 个周期，而完整的一次切换通常需要 99-105 个周期^[94-95]。与 MPK 类似的技术 PKS 具有类似的开销。遗憾的是，它们的保护数量受到页表可用位的限制。Moat 实现了现代高效隔离硬件类似的开销，但 Moat 不受硬件约束。其它的虚拟化隔离方案将会带来更大的开销，比如，基于 EPT 的隔离一次调用需要 396 个周期^[31,94-95]。

5.2.2 隔离设备的开销

Moat 支持低开销的隔离。在基于语言机制实现隔离的方法中，隔离的开销主要由两个部分构成：故障隔离代理生成的故障捕获代码和热更新代理执行的任务统计以

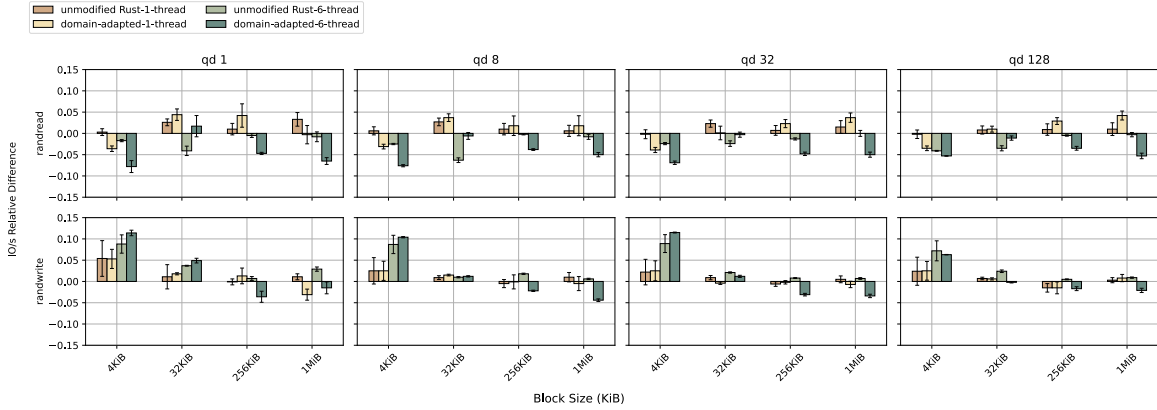


图 5.1 null block 性能测试

及共享堆数据所有权转移。在 Linux 系统中，还包含了使用接口重定向 3.7 带来的转换开销。为了了解 Rust 的安全性是否适用于开发现代操作系统内核中的高性能驱动程序，本文测量了 Linux 中的两个设备驱动程序的隔离开销：null block 和 Nvme，每个驱动使用不同的存储介质。null block 使用内存模拟磁盘，因此其性能不受设备本身的限制，而 nvme 驱动则使用非易失性存储介质，其性能易受到设备运行状态的影响。

在两个实验中，均使用 fio 来生成读写请求。为了为评估提供一个最佳基线，本文设定了 fio 的配置参数从而为读写提供最小的延迟，以便更好强调隔离的开销。具体来说，本文使用 fio 的 io_uring 引擎来提交多个请求，设置直接 I/O 标志以绕过页面缓存，并使用轮询模式。本文将改造为域的驱动与未被修改的 Rust 驱动和 C 实现的驱动进行比较（下文统一称为域驱动，Rust 驱动，C 驱动），计算域驱动和 Rust 驱动相对于 C 驱动的性能优劣。

null block 测试 null block 设备使用系统内存作为后备存储介质。由于其需要使用内核内存，在测试前，首先通过写磁盘提前分配内存以消除内核内存分配造成的影响。本文在不同的队列深度 (qd)、线程数和块大小配置 (block size) 下执行多次随机读写测试（图 5.1）。就从使用 Rust 实现设备驱动的角度看，在随机读测试中，Rust 驱动的总体性能与 C 驱动类似，吞吐量最多分别降低 4.1% 和提高 3.3%；在随机写中，Rust 驱动的总体性能优于 C 驱动，本文检查了 Rust 驱动和 C 驱动的具体实现，C 驱动在处理写请求中实现了更多的功能，比如检查是否使用缓存以及修改页标记等，本文将 Rust 驱动的优势归功于更短的代码路径，尽管如此，仍然可以得出结论，使用 Rust 实现高性能驱动是可行的。修改后的域驱动与 Rust 驱动相比，两者的性能在不同的配置下各有优略。在随机读测试中，32 个不同配置下有 18 个配置域驱动的性能

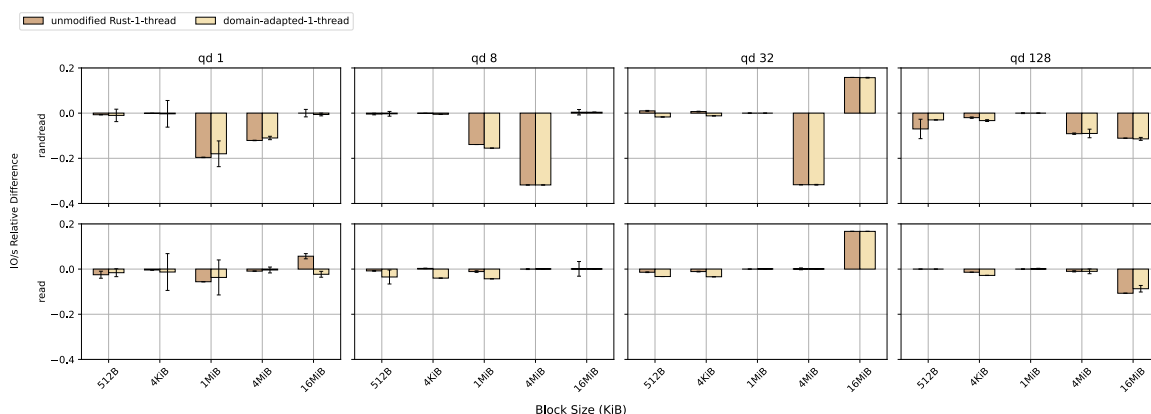


图 5.2 Nvme 性能测试

能发生了下降 (1.3%-6.2%)，在随机写测试中，19 个配置下域驱动的性能发生了下降 (0.1%-5%)，就整体而言，在随机读测试下，域驱动带来了平均 1.2% 的开销，在随机写测试下则带来了平均 0.9% 的开销。这说明为支持隔离而额外引入的代码并没有显著降低程序的性能。

Nvme 测试 在 nvme 设备上的结果如图 5.2 所示，本文没有与 null 块设备一样在多种配置下测量随机写的性能，因为 nvme 设备的写性能会严重受到设备因素的干扰，鉴于时间有限，本文还没有在 Nvme 驱动程序中添加多线程的支持，因此删去了多线程的测试。在大多数配置下，Rust 驱动同样获得了与 C 驱动相近的性能。当块大小增大到 1MB-16MB，Rust 的性能波动会突然变得很大，在 $qd=32, blk_size=4MB$ 的配置下，Rust 驱动的性能下降了 32%，在 $qd=32, blk_size=16MB$ 配置下，Rust 驱动的性能反而提升了 15%。本文检查了两个驱动的实现，并没有发现可能与这些变化相关的的实现差异，本文将对此更进一步的分析留作未来的工作。在引入隔离的情况下，域驱动与 Rust 驱动几乎没有差异，随机读的平均开销可以忽略不计 (0.09%)，顺序读的平均开销为 0.9%。

本文单独测量了随机写的性能差异，图 5.3 显示了完整写完一次磁盘的吞吐量变化，由于受到运行时间和设备缓存的影响，三者在前 30s 内性能都产生了不同程度的波动，但差距保持在 2% 以内，在经过 30s 左右的高速写入后，设备性能快速下降并达到稳态，此时三者的总体性能几乎一样。也就是说，隔离带来的开销相比硬件设备本身的开销，可以忽略不计。

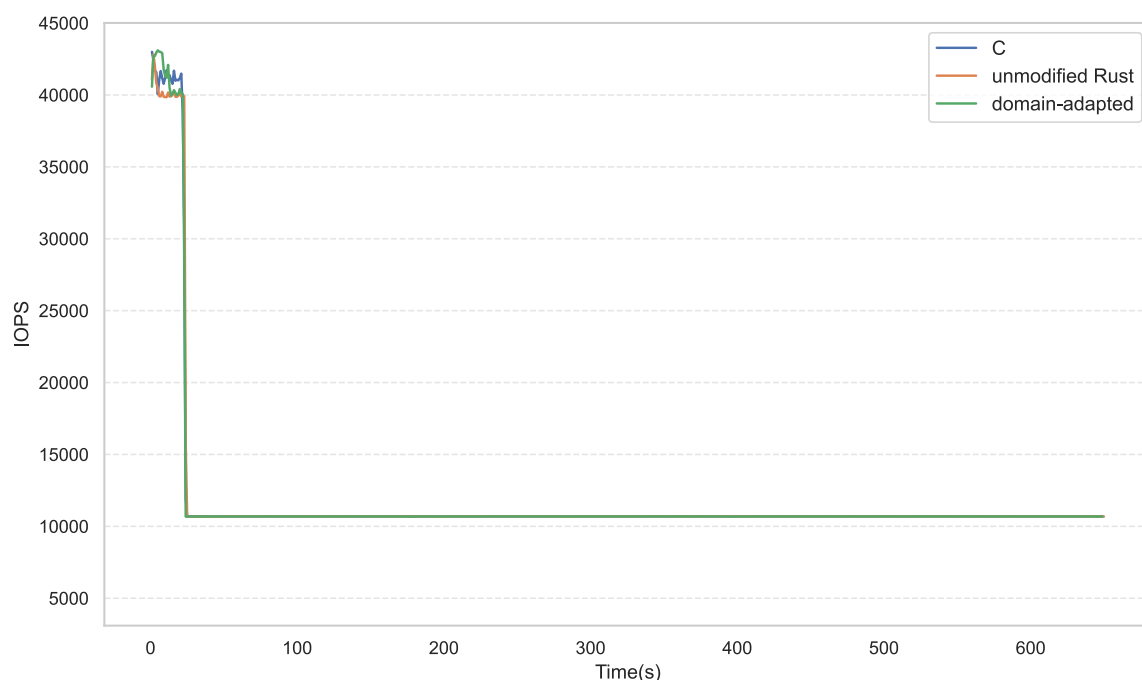


图 5.3 Nvme 写性能测试

5.2.3 跨域通信开销

随着较大的内核子系统被分解为多个隔离域，其中一些隔离域的实现就需要依赖其它隔离域的功能。同时，鉴于对安全性要求的提高，原来可能位于一个域中的功能也会被拆分到其它域中。在这两种情况下，域间通信就会变得更加频繁。为了评估任务在一次请求中执行多次域间调用而产生的开销，设计了这个实验。首先，在块设备驱动域中实现了一个函数，用于对缓冲区进行随机填充，然后，额外实现一个虚拟的隔离域，并且在域中实现缓冲区随机填充的功能。在驱动域中，在进行真正的读写前，以两种不同的方式调用缓冲区随机填充功能：(1) 调用虚拟域的接口，这相当于在隔离模式进行域间调用；(2) 直接调用函数实现，这等价于传统的没有进行隔离划分的情况。

本文针对 512B 和 4KB 块大小的请求调用缓冲区随机填充，然后通过重复调用来模拟不同数量的域间调用或者函数调用，并计算隔离情况下相对于无隔离的性能比例。本文在 null block 驱动上统计了吞吐量，实验结果如图 5.4 所示。在 512B 块大小的请求下，随着域调用次数的增加，吞吐量占比逐渐下降，其中，单次域调用导致吞吐量下降 1.5%，而执行 8 次域调用吞吐量下降了 6%。开销的来源是显而易见的，首先，512B 缓冲区的随机填充所需要的时钟周期很小，小于因为隔离需要而引入的

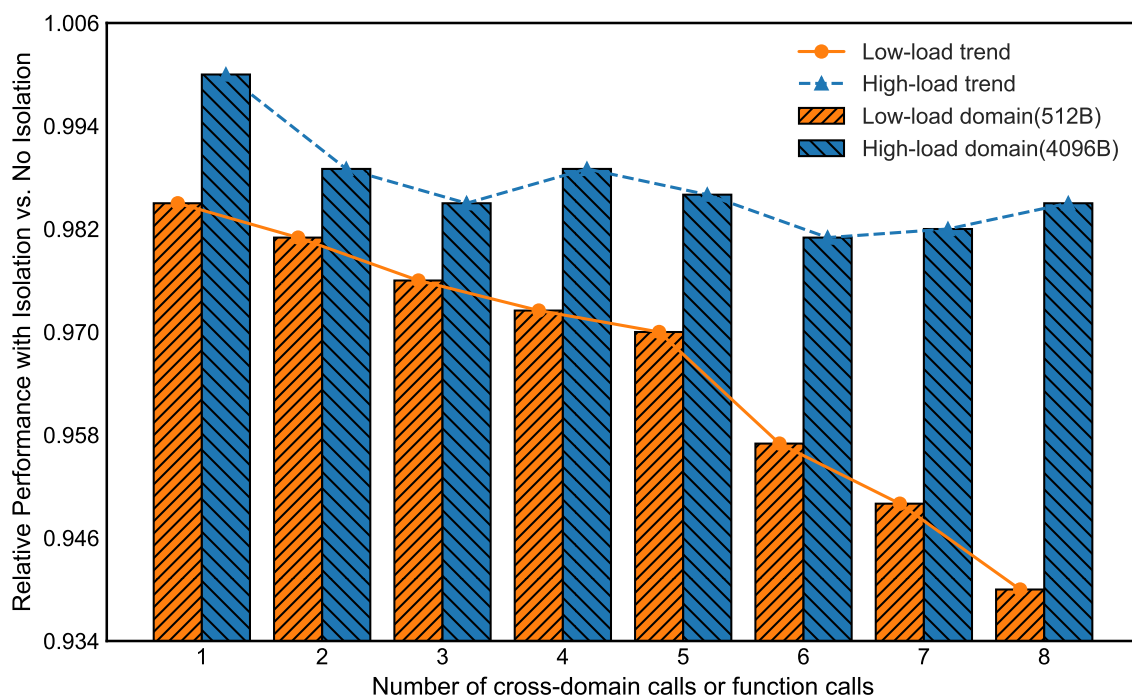


图 5.4 不同的跨域调用次数和域内工作负载对性能的影响

额外代码，其次，直接函数调用给编译器提供了函数内联的机会。4KB 块大小的结果也验证了这个结论。当虚拟域执行 4KB 大小缓冲区的随机填充时，单次域调用和 8 次域调用的吞吐量分别下降了下降了 0.01% 和 1.5%，远小于 512B 的情况，因为此时随机填充占据了更多的 CPU 时间。

对于隔离域来说，512B 和 4KB 的缓冲区随机填充本质上代表了域内执行的不同大小的工作负载。这也反过来给开发者进行域划分提供了一些很有价值的反馈，即尽量避免将较小的功能划分到单个域中，而是尽可能让独立的域承担一定的工作量，从而在安全性要求和性能之间取得折衷。

5.3 故障恢复的开销

Moat 的故障隔离对许多需要保证可靠性的服务至关重要。一方面，正确的故障处理有助于确保内核模块不会发生资源泄露或死锁从而停止运行；另一方面，借助故障隔离代理，可以对用户隐藏模块崩溃的发生并透明地进行调用重启。但是如果故障隔离的开销过大，以至于和用户程序直接暂停服务导致的结果一样，那故障隔离就失去了意义。

因此，为了评估故障恢复对用户程序性能的影响，本文开发了一个测试，在设备

表 5.2 故障隔离的内存开销

Type	rnull	rnull(LTO)	rnmve	rnmve(LTO)
Text	68.00 KB	68.00 KB	136.00 KB	132.00 KB
Data	8.00 KB	4.00 KB	12.00 KB	12.00 KB
Bss	4.00 KB	4.00 KB	4.00 KB	4.00 KB
Rodata	12.00 KB	8.00 KB	16.00 KB	16.00 KB
.rela.dyn	6.19 KB	4.90 KB	11.88 KB	12.52 KB
.eh_frame	10.18 KB	6.75 KB	20.78 KB	12.57 KB
.eh_frame_hdr	2.13 KB	1.21 KB	3.75 KB	2.10 KB
.gcc_except_table	3.93 KB	3.54 KB	8.78 KB	7.56 KB
Total (unwind)	114.43 KB (14.2%)	100.40 KB (11.4%)	213.19 KB (15.6%)	198.75 KB (11.2%)

驱动域中定时地触发错误来模拟内核模块在运行过程中可能的故障情况，这会触发故障恢复流程。实验调整了错误触发的间隔，以确保重启的调用不会再次触发错误，然后使用 FIO 作为基准应用程序并按照 4KB 块大小连续读取块设备。实验设置了不同大小的错误发生间隔，结果如图 5.5 所示。故障恢复会使得 I/O 吞吐量下降。在无故障和在 100us 故障频率下，读取的吞吐量分别为 480k/s 和 418k/s，性能下降了 13%，吞吐量随着故障频率的升高而降低。

实验统计了故障恢复花费的时间为 20-25us，因为故障恢复需要执行堆栈展开并释放分配的资源，而这需要解析和扫描大量的 DWARF 信息。相较而言，在本文的设置中，完成单个 I/O 请求的时间平均为 2us，因此故障频率的升高会导致更多的 I/O 请求遭受故障恢复的影响。在没有隔离的情况下，故障将会直接导致用户程序崩溃，相比之下，即使在 100us 这样高频率下，故障恢复带来的开销依然在可以接受的范围。

故障隔离的内存开销 故障隔离在时间上的开销可以接受，本文统计了内核模块的大小，以确认故障隔离是否会在空间上带来较大的开销。实验统计了两个域驱动程序，并且对需要加载到内存中的不同部分进行了区分。结果如表 5.2 所示，其中 .eh_frame、.eh_frame_hdr、.gcc_except_table 是实现故障隔离所需的部分，.eh_frame_hdr 是为了在堆栈展开期间加速函数查找而生成的，即使去掉也不影响。在不开启更高级别优化的情况下，这三部分内容分别占到了整个可执行文件大小的 14.2% 和 15.6%。链接时优化（LTO）是一种整体程序优化技术，可以提高程序的运行速度并减小可执行

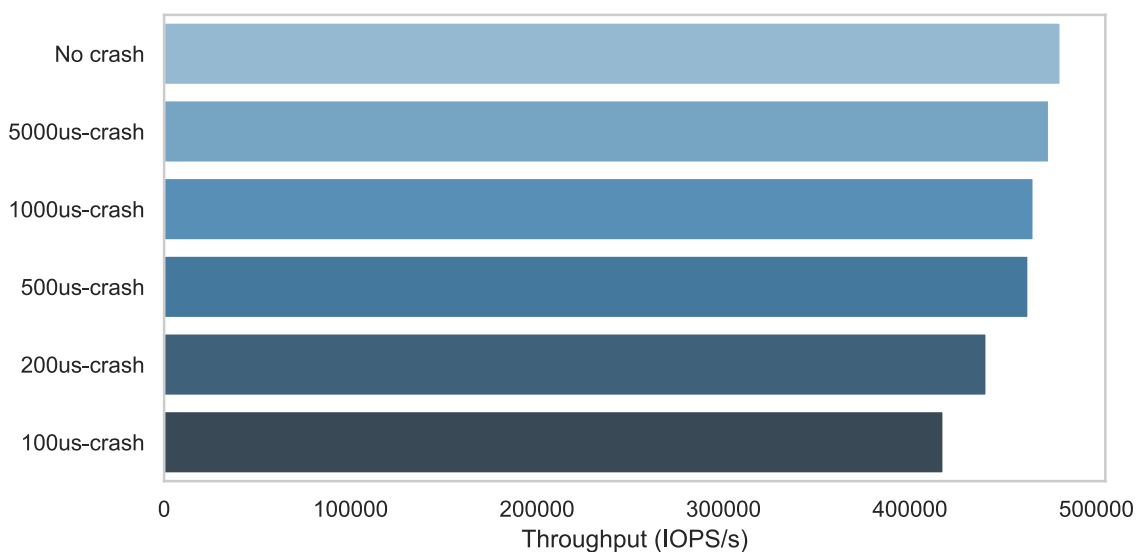


图 5.5 不同频率的故障发生次数对性能的影响

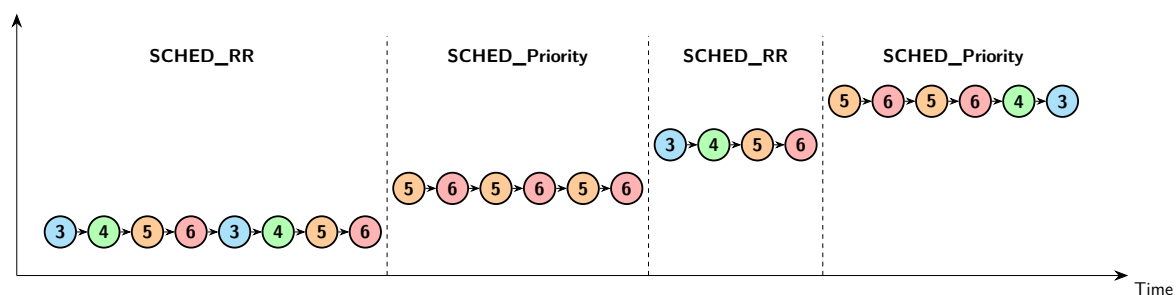


图 5.6 调度器的热更新

文件的大小。在 LTO 优化下，整个可执行文件大小分别减小了 12.3% 和 6.7%，而这三部分的大小也因为优化而减小，分别只占到整个文件大小的 11.4% 和 11.2%。本文认为实现故障隔离所导致的额外空间开销相比其带来的好处是可以接受的。

5.4 热更新

在这个实验中，本文评估隔离模块的热更新能力。由于暂时无法使用 Rust 实现 Linux 的核心模块，因此，本文使用案例研究（如 Baumann 等人^[96]）在 AlienOS 中进行实验。实验以调度器模块的动态替换为例，验证 Moat 在实施核心模块热更新方面的可行性与有效性。之所以选择这个模块，是因为现代操作系统的调度器通常无法满足用户特定的工作负载，而由于与内核的结构和其它模块高度耦合，对调度器的升级的大多数情况下都需要重启系统。

实验设计了包含多个用户任务的场景，这些任务执行矩阵计算，其中任务 5 和 6

表 5.3 Moat 的热更新服务暂停时间

Phase	Stateful*	Stateless*
Total Time	1070 μ s	760 μ s
Load new Domain	324 μ s	300 μ s
Service Downtime	146 μ s	$\times$
Task Synchronization	134 μ s	$\times$
Initialization/State Migration	12 μ s	122ns
Domain Replacement	347ns	200ns
RCU Synchronization	$\times$	27 μ s
Resources Cleanup	576 μ s	410 μ s
Other	24 μ s	23 μ s

被赋予最高优先级 (Priority=16)，其余任务为普通优先级 (Priority=0)，初始调度策略为实时轮转调度 (RR)，任务按轮转顺序平等获取 CPU 时间。通过触发四次热更新操作 (RR $\leftrightarrow$ Priority 循环切换)，观测系统在升级过程中的行为：如图 5.6 所示，当切换至优先级调度策略时，高优先级任务 5 和 6 立即抢占 CPU 资源，并且会一直占有 CPU，其余任务进入等待队列；而回退至 RR 策略后，所有任务恢复公平轮转，且历史运行进度（如任务的矩阵计算迭代计数）在升级前后完全一致，未出现状态丢失或计算重复。在整个测试运行过程中，本文没有重启系统。实验结果表明，Moat 能够处理组件级别更新。

服务暂停时间 热更新的真正作用在于减少系统服务暂停的时间，本实验统计了在 Linux 系统上更新一个有状态域和无状态域的时间分布。与现有的实施热更新的作品不同，在 Moat 中，隔离模块的热更新建立在基于语言机制的隔离上，并使用特殊的状态保存和同步机制，这使得该方法当前只能针对 Rust 开发的模块，因此，很难将 Moat 的热更新能力与其它方法进行直接对比，因此本文独立分析 Moat 的方法与其它方法进行理论上的比较。

作为示例，表 5.3 显示了使用 Moat 进行热更新的时间分解。实验使用块设备驱动域和一个无状态日志输出域作为典型的测试用例。实验将热更新划分为多个阶段，有状态和无状态略有区别，其中有状态的热更新包含任务状态同步阶段而不包含 RCU 同步，无状态的热更新相反之，Task sync、Domain reinit and State Transfer 和 Domain

replace 构成了升级过程中的服务暂停时间。正如所观察到的, Moat 能够在 1ms 左右的时间更新一个有状态域。值得注意的是, 在热更新的大多数阶段, 例如新域加载, 资源清理, 正在运行的任务不会受到影响。在服务暂停阶段, 只有涉及到被更新的域的任务会暂停执行, 服务暂停时间仅仅占整个过程的 14%(146us)。这归功于 Moat 中数据状态的分离存储和高效的任务统计和检查方法。对于无状态域, 整个升级期间都不会中断当前正在运行的服务, 因此不存在服务暂停时间, 其花费的大多数时间是用于新域加载和资源清理。根据 Plugshed 的报告, 使用 kpatch^[11] 升级一个内核模块导致的服务暂停时间在 2800-3000us 左右, 这比 Moat 高出了 20 倍, 原因在于其需要暂停系统中的全部 CPU 并且扫描系统中的所有任务的堆栈, 并且 kpatch 还不支持组件级别的热更新。

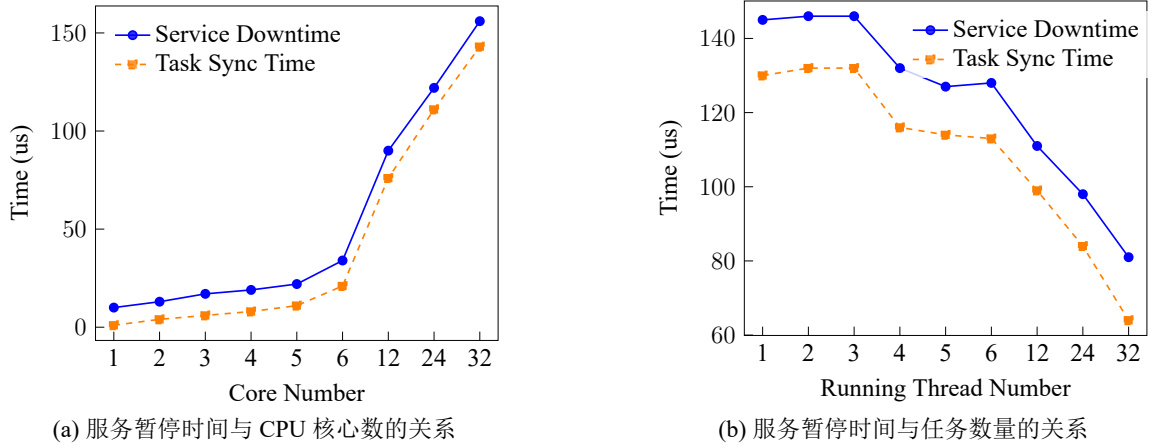


图 5.7 服务暂停时间分析

热更新的可扩展性 在 Moat 中, 热更新的时间跟系统中的 CPU 密切相关, 因为在统计位于域中的任务数量之前, 需要向其它 CPU 发送一次 IPI。在本实验中, 本文以不同核心数的块设备驱动热更新为例, 展示 Moat 的热更新和 CPU 数量的关系。为了最大限度地减少活动工作负载的影响, 在测量期间仅运行执行热更新的程序。如图 5.7a 所示, 实验通过 CPU 热插拔关闭系统可用的 CPU, 当系统中可用 CPU 数量从 1 增长到 32 时, 用于任务状态同步的时间从 1us 增长到 143us, 服务暂停时间从 10us 增长到 156us, 整个过程服务暂停时间中用于 Domain reinit 和 State transfer, Domain replace 的部分几乎保持在 9-14us。

对于任务状态同步来说, CPU 核心数量的增加意味着 IPI 数量的增加, 从而增加其花费时间。在实验中, 本文观察到任务状态同步所增长的时间与 CPU 核心增长的数量并非完全线性相关, 产生这个的原因是一个微妙细节: 如果 CPU 核心此时位于

用户态，那它因为 IPI 中断所花费的时间 (4-5us) 就比当前处于内核态中的 CPU(2-4us) 多，因为位于用户态的 CPU 核心在处理中断的过程中需要进行特权级切换并保存更多上下文，而那些位于内核态的 CPU 也会根据是否处于 idle 状态而花费不同的时间。

为了进一步了解活跃工作负载对块设备驱动程序实时更新的影响，本文在使能所有 CPU 核心的条件下，逐步增加更多的任务读取该设备的数据。当扩展访问域的并发任务数量时，会出现一个违反直觉的现象（图 5.7b）：随着任务数量从 1 增加到 32，同步时间从 130 μ 秒减少到 64 μ 秒。出现这种反转的原因是，随着任务数量的增加，因为工作负载的缘故，更多 CPU 核心将会有更大的概率处于内核态中，从而通过避免特权级转换来降低 IPI 处理延迟，进而降低任务状态同步的时间。

虽然这些结果是在受控的实验条件下获得的，但实际部署在不同工作负载下，服务停机时间始终保持在 100 到 200 μ s 之间。这种稳定性归功于 Moat 的双阶段设计：时间关键型操作（状态转移和替换）受确定性延迟约束，而成本可变的阶段（例如任务状态同步）则通过统计优化动态适应系统状态。这种架构分离确保了即使在硬件扩展挑战下也能实现可预测的最坏情况行为。

5.5 本章小结

在本章中，本文评估了 Moat 的域隔离抽象开销，相较于同样使用语言机制实现隔离的 Redleaf，Moat 大幅降低了跨域调用的开销，隔离的 Linux 驱动程序平均 I/O 吞吐量仅下降 1% 到 2%。Moat 的故障恢复造成了少量空间开销，但相较于其带来的好处，这是合理的。最后，本文评估了 Moat 的实时更新机制，与隔离机制的有效结合使得实时更新具有良好的表达性和更短的服务暂停时间，实时更新导致的服务停机时间仅为 100 到 200 微秒。

结论

本文介绍了 Moat, 一种使用语言机制实现内核模块隔离和热升级的方法, 用于提高内核的安全性、可靠性和可扩展性。在 Rust 语言提供的安全保证基础上, 基于系统分解和模块化思想, Moat 提供了一个通用的用于实施模块隔离的框架, Moat 利用私有堆和共享堆的设计来实现模块间的内存隔离和通信, Moat 还通过故障隔离加强了内存隔离的有效性。在模块隔离的基础上, Moat 提供了对隔离模块的组件级热更新支持, 这依赖 Moat 提出的状态分离存储和高效的状态同步方法。本文通过实现一个原型操作系统 Alien OS 来验证这个方法, 更重要的是, 本文将这个方法应用在现代商用操作系统 Linux 上。实验表明, Moat 是一个通用的架构, 其隔离具有很小的开销, Linux 隔离驱动的 I/O 吞吐量平均仅下降 1%-2%, Moat 的组件级热升级导致的服务暂停时间很短, 在 100-200us 左右。未来, 在已经实施的基础设施上, 本文希望可以为更多的 Linux 内核模块实施基于语言机制的隔离和热升级。

参考文献

- [1] StackScale. Linux Kernel Surpasses 40 Million Lines of Code[EB/OL]. 2025. <https://www.stackscal e.com/blog/linux-kernel-surpasses-40-million-lines-code/>.
- [2] Li Z, Wang J, Sun M, et al. Securing the Device Drivers of Your Embedded Systems: Framework and Prototype[C]. ARES '19: Proceedings of the 14th International Conference on Availability, Reliability and Security. Canterbury, CA, United Kingdom: Association for Computing Machinery, 2019.
- [3] Chen X, Li Z, Zhang J, et al. Veld: Verified Linux Drivers[C]. KISV '24: Proceedings of the 2nd Workshop on Kernel Isolation, Safety and Verification. Austin, TX, USA: Association for Computing Machinery, 2024: 23-30.
- [4] Li J, Miller S, Zhuo D, et al. An incremental path towards a safer OS kernel[C]. HotOS '21: Proceed- ings of the Workshop on Hot Topics in Operating Systems. Ann Arbor, Michigan: Association for Computing Machinery, 2021: 183-190.
- [5] Miller S, Zhang K, Zhuo D, et al. Practical Safe Linux Kernel Extensibility[C]. HotOS '19: Proceedings of the Workshop on Hot Topics in Operating Systems. Bertinoro, Italy: Association for Computing Machinery, 2019: 170-176.
- [6] Hat R. CVE-2021-43267[EB/OL]. 2025. <https://access.redhat.com/security/cve/cve-2021-43267>.
- [7] Narayanan V, Balasubramanian A, Jacobsen C, et al. LXDs: Towards Isolation of Kernel Subsystems [C]. 2019 USENIX Annual Technical Conference (USENIX ATC 19). Renton, WA: USENIX Asso- ciation, 2019: 269-284.
- [8] Lefeuvre H, Bădoiu V A, Jung A, et al. Flexos: Towards flexible os isolation[C]. Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Oper- ating Systems. 2022: 467-482.
- [9] Von Eicken T, Chang C C, Czajkowski G, et al. J-Kernel: A capability-based operating system for Java [J]. Secure Internet programming: security issues for mobile and distributed objects, 1999: 369-393.
- [10] Zhou F, Condit J, Anderson Z, et al. SafeDrive: safe and recoverable extensions using language- based techniques[C]. OSDI '06: Proceedings of the 7th Symposium on Operating Systems Design and Implementation. Seattle, Washington: USENIX Association, 2006: 45-60.
- [11] Kpatch. kpatch: dynamic kernel patching[EB/OL]. 2025. <https://github.com/dynup/kpatch>.
- [12] Kernel L. Livepatch —The Linux Kernel documentation[EB/OL]. 2025. <https://www.kernel.org/do c/html/latest/livepatch/livepatch.html>.
- [13] Chen H, Yu J, Chen R, et al. POLUS: A POverful Live Updating System[C]. ICSE '07: Proceedings of the 29th International Conference on Software Engineering. USA: IEEE Computer Society, 2007:

- 271-281.
- [14] Arnold J, Kaashoek M F. Ksplice: automatic rebootless kernel updates[C]. EuroSys '09: Proceedings of the 4th ACM European Conference on Computer Systems. Nuremberg, Germany: Association for Computing Machinery, 2009: 187-198.
 - [15] Ma T, Chen S, Wu Y, et al. Efficient Scheduler Live Update for Linux Kernel with Modularization [C]. ASPLOS 2023: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. Vancouver, BC, Canada: Association for Computing Machinery, 2023: 194-207.
 - [16] Giuffrida C, Kuijsten A, Tanenbaum A S. Safe and automatic live update for operating systems[C]. ASPLOS '13: Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems. Houston, Texas, USA: Association for Computing Machinery, 2013: 279-292.
 - [17] Zhang X, Zheng X, Wang Z, et al. Fast and Scalable VMM Live Upgrade in Large Cloud Infrastructure[C]. ASPLOS '19: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. Providence, RI, USA: Association for Computing Machinery, 2019: 93-105.
 - [18] Humphries J T, Natu N, Chaugule A, et al. ghOS: Fast & Flexible User-Space Delegation of Linux Scheduling[C]. SOSP '21: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. Virtual Event, Germany: Association for Computing Machinery, 2021: 588-604.
 - [19] Russinovich M, Govindaraju N, Raghuraman M, et al. Virtual machine preserving host updates for zero day patching in public cloud[C]. EuroSys '21: Proceedings of the Sixteenth European Conference on Computer Systems. Online Event, United Kingdom: Association for Computing Machinery, 2021: 114-129.
 - [20] Bell D E, La Padula L J. Secure computer system: Unified exposition and multics interpretation[J]. (No Title), 1976.
 - [21] Fraim L J. SCOMP: A solution to the multilevel security problem[J]. Computer, 1983, 16(07): 26-34.
 - [22] Mehrab A K M F, Nikolaev R, Ravindran B. Kite: lightweight critical service domains[C]. EuroSys '22: Proceedings of the Seventeenth European Conference on Computer Systems. Rennes, France: Association for Computing Machinery, 2022: 384-401.
 - [23] Boyd-Wickizer S, Zeldovich N. Tolerating malicious device drivers in Linux[C]. USENIXATC'10: Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference. Boston, MA: USENIX Association, 2010: 9.
 - [24] David F M, Chan E M, Carlyle J C, et al. CuriOS: improving reliability through operating system structure[C]. OSDI'08: Proceedings of the 8th USENIX Conference on Operating Systems Design

- and Implementation. San Diego, California: USENIX Association, 2008: 59-72.
- [25] Chen X, Li Z, Jain T, et al. Limitations and Opportunities of Modern Hardware Isolation Mechanisms [C]. 2024 USENIX Annual Technical Conference (USENIX ATC 24). Santa Clara, CA: USENIX Association, 2024: 349-368.
- [26] Sung M, Olivier P, Lankes S, et al. Intra-unikernel isolation with Intel memory protection keys[C]. VEE '20: Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. Lausanne, Switzerland: Association for Computing Machinery, 2020: 143-156.
- [27] Swift M M, Bershad B N, Levy H M. Improving the reliability of commodity operating systems[J]. SIGOPS Oper. Syst. Rev., 2003, 37(5): 207-222.
- [28] Lee D, Kohlbrenner D, Shinde S, et al. Keystone: an open framework for architecting trusted execution environments[C]. EuroSys '20: Proceedings of the Fifteenth European Conference on Computer Systems. Heraklion, Greece: Association for Computing Machinery, 2020.
- [29] Jia Y, Liu S, Wang W, et al. HyperEnclave: An Open and Cross-platform Trusted Execution Environment[C]. 2022 USENIX Annual Technical Conference (USENIX ATC 22). Carlsbad, CA: USENIX Association, 2022: 437-454.
- [30] Gravani S, Hedayati M, Criswell J, et al. Fast Intra-kernel Isolation and Security with IskiOS[C]. RAID '21: Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses. San Sebastian, Spain: Association for Computing Machinery, 2021: 119-134.
- [31] Narayanan V, Huang Y, Tan G, et al. Lightweight kernel isolation with virtualization and VM functions[C]. VEE '20: Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. Lausanne, Switzerland: Association for Computing Machinery, 2020: 157-171.
- [32] Sartakov V A, Vilanova L, Pietzuch P. CubicleOS: a library OS with software componentisation for practical isolation[C]. ASPLOS '21: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. Virtual, USA: Association for Computing Machinery, 2021: 546-558.
- [33] Haas A, Rossberg A, Schuff D L, et al. Bringing the web up to speed with WebAssembly[C]. Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation. 2017: 185-200.
- [34] Castro M, Costa M, Martin J P, et al. Fast byte-granularity software fault isolation[C]. SOSP '09: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles. Big Sky, Montana, USA: Association for Computing Machinery, 2009: 45-58.

- [35] Erlingsson Ú, Abadi M, Vrabie M, et al. XFI: software guards for system address spaces[C]. OSDI '06: Proceedings of the 7th Symposium on Operating Systems Design and Implementation. Seattle, Washington: USENIX Association, 2006: 75-88.
- [36] Mao Y, Chen H, Zhou D, et al. Software fault isolation with API integrity and multi-principal modules [C]. SOSP '11: Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles. Cascais, Portugal: Association for Computing Machinery, 2011: 115-128.
- [37] Sehr D, Muth R, Biffle C, et al. Adapting software fault isolation to contemporary CPU architectures [C]. USENIX Security'10: Proceedings of the 19th USENIX Conference on Security. Washington, DC: USENIX Association, 2010: 1.
- [38] Zhong Y, Li H, Wu Y J, et al. XRP: In-Kernel Storage Functions with eBPF[C]. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). Carlsbad, CA: USENIX Association, 2022: 375-393.
- [39] Huang Y, Narayanan V, Detweiler D, et al. KSplit: Automating Device Driver Isolation[C]. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). Carlsbad, CA: USENIX Association, 2022: 613-631.
- [40] Klein G, Elphinstone K, Heiser G, et al. seL4: formal verification of an OS kernel[C]. SOSP '09: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles. Big Sky, Montana, USA: Association for Computing Machinery, 2009: 207-220.
- [41] Herder J N, Bos H, Gras B, et al. MINIX 3: a highly reliable, self-repairing operating system[J]. SIGOPS Oper. Syst. Rev., 2006, 40(3): 80-89.
- [42] Chen H, Miao X, Jia N, et al. Microkernel Goes General: Performance and Compatibility in the HongMeng Production Microkernel[C]. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). Santa Clara, CA: USENIX Association, 2024: 465-485.
- [43] Back G, Hsieh W C. The KaffeOS Java runtime system[J]. ACM Trans. Program. Lang. Syst., 2005, 27(4): 583-630.
- [44] Boos K, Liyanage N, Ijaz R, et al. Theseus: an Experiment in Operating System Structure and State Management[C]. 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, 2020: 1-19.
- [45] Narayanan V, Huang T, Detweiler D, et al. RedLeaf: Isolation and Communication in a Safe Operating System[C]. 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, 2020: 21-39.
- [46] Hunt G C, Larus J R. Singularity: rethinking the software stack[J]. SIGOPS Oper. Syst. Rev., 2007, 41(2): 37-49.

- [47] Peng Y, Tian H, Junyang Z, et al. Asterinas: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB[EB/OL]. 2025. <https://arxiv.org/abs/2506.03876>.
- [48] Emmerich P, Ellmann S, Bonk F, et al. The Case for Writing Network Drivers in High-Level Programming Languages[C]. 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS). 2019: 1-13.
- [49] Developers T R P. The Borrow Checker - Rust Compiler Development Guide[Z]. https://rustc-dev-guide.rust-lang.org/borrow_check.html. 2025.
- [50] WG R S C. RustSec Advisory Database[Z]. <https://github.com/RustSec/advisory-db>. 2024.
- [51] Developers T R P. Behavior Considered Undefined - The Rust Reference[Z]. <https://doc.rust-lang.org/nightly/reference/behavior-considered-undefined.html>. 2025.
- [52] Yang J, Hawblitzel C. Safe to the last instruction: automated verification of a type-safe operating system[J]. SIGPLAN Not., 2010, 45(6): 99-110.
- [53] Cutler C, Kaashoek M F, Morris R T. The benefits and costs of writing a POSIX kernel in a high-level language[C]. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). Carlsbad, CA: USENIX Association, 2018: 89-105.
- [54] Levy A, Campbell B, Ghena B, et al. Multiprogramming a 64kB Computer Safely and Efficiently[C]. SOSPP '17: Proceedings of the 26th Symposium on Operating Systems Principles. Shanghai, China: Association for Computing Machinery, 2017: 234-251.
- [55] Rotem E, Yoaz A, Rappoport L, et al. Intel Alder Lake CPU Architectures[J]. IEEE Micro, 2022, 42(3): 13-19.
- [56] Stephens N, Biles S, Boettcher M, et al. The ARM Scalable Vector Extension[J]. IEEE Micro, 2017, 37(2): 26-39.
- [57] Guz Z, Li H (, Shayesteh A, et al. NVMe-over-fabrics performance characterization and the path to low-overhead flash disaggregation[C]. SYSTOR '17: Proceedings of the 10th ACM International Systems and Storage Conference. Haifa, Israel: Association for Computing Machinery, 2017.
- [58] Wei X, Cheng R, Yang Y, et al. Characterizing Off-path SmartNIC for Accelerating Distributed Systems[C]. 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23). Boston, MA: USENIX Association, 2023: 987-1004.
- [59] Project C. Memory Safety[EB/OL]. 2025. <https://www.chromium.org/Home/chromium-security/memory-safety/>.
- [60] Microsoft. A Proactive Approach to More Secure Code[EB/OL]. 2025. <https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>.
- [61] Langui. Memory Unsafety in Apple's Operating Systems[EB/OL]. 2025. <https://langui.sh/2019/07/23/apple-memory-safety/>.

- [62] Corporation I. Intel(R) 64 and IA-32 Architectures Software Developer’s Manual[EB/OL]. 2016. <https://software.intel.com/en-us/articles/intel-sdm>.
- [63] Arm. Introduction to the Memory Tagging Extension[EB/OL]. 2025. <https://developer.arm.com/documentation/108035/0100/Introduction-to-the-Memory-Tagging-Extension>.
- [64] Liljestrand H, Nyman T, Gunn L J, et al. PACStack: an Authenticated Call Stack[C]. 30th USENIX Security Symposium (USENIX Security 21). USENIX Association, 2021: 357-374.
- [65] Righini M. Enabling Intel virtualization technology features and benefits[J]. Intel Corporation, White paper, 2010.
- [66] AMD. AMD Virtualization Technology[EB/OL]. 2025. <https://www.amd.com/en/solutions/hci-and-virtualization.html>.
- [67] Pinto S, Santos N. Demystifying Arm TrustZone: A Comprehensive Survey[J]. ACM Comput. Surv., 2019, 51(6).
- [68] Ganz J, Peisert S. ASLR: How Robust Is the Randomness?[C]. 2017 IEEE Cybersecurity Development (SecDev). 2017: 34-41.
- [69] Dang T H, Maniatis P, Wagner D. The Performance Cost of Shadow Stacks and Stack Canaries[C]. ASIA CCS ’15: Proceedings of the 10th ACM Symposium on Information, Computer and Communications Security. Singapore, Republic of Singapore: Association for Computing Machinery, 2015: 555-566.
- [70] Narayan S, Garfinkel T, Johnson E, et al. Segue & ColorGuard: Optimizing SFI Performance and Scalability on Modern Architectures[C]. ASPLOS ’25: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. Rotterdam, Netherlands: Association for Computing Machinery, 2025: 987-1002.
- [71] Yedidia Z. Lightweight Fault Isolation: Practical, Efficient, and Secure Software Sandboxing[C]. ASPLOS ’24: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. La Jolla, CA, USA: Association for Computing Machinery, 2024: 649-665.
- [72] Intel. <https://www.dpdk.org/>. 2021.
- [73] Vangoor B K R, Tarasov V, Zadok E. To {FUSE} or not to {FUSE}: Performance of {User-Space} file systems[C]. 15th USENIX Conference on File and Storage Technologies (FAST 17). 2017.
- [74] Google. Eliminating Memory Safety Vulnerabilities in Android[EB/OL]. 2025. <https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html>.
- [75] Microsoft. C# Language[EB/OL]. 2025. <https://dotnet.microsoft.com/en-us/languages/csharp>.
- [76] Google. Go Programming Language[EB/OL]. 2025. <https://go.dev/>.

- [77] Crichton W, Gray G, Krishnamurthi S. A Grounded Conceptual Model for Ownership Types in Rust [J]. Proceedings of the ACM on Programming Languages, 2023, 7: 1224-1252.
- [78] Li J, Lattuada A, Zhou Y, et al. Linear types for large-scale systems verification[J]. Proceedings of the ACM on Programming Languages, 2022, 6(OOPSLA1): 1-28.
- [79] Klabnik S, Nichols C. The Rust programming language[EB/OL]. 2025. <https://doc.rust-lang.org/book/>.
- [80] Hoare C A R. An axiomatic basis for computer programming[J]. Commun. ACM, 1969.
- [81] Developers R P. Redox - Your Next(Gen) OS[EB/OL]. 2025. <http://www.redox-os.org/>.
- [82] Lankes S, Breitbart J, Pickartz S. Exploring Rust for Unikernel Development[C]. PLOS '19: Proceedings of the 10th Workshop on Programming Languages and Operating Systems. Huntsville, ON, Canada: Association for Computing Machinery, 2019: 8-15.
- [83] OS D. Drone OS[EB/OL]. 2025. <https://www.drone-os.com/>.
- [84] For Linux R. Rust for Linux[EB/OL]. 2025. <https://rust-for-linux.com/>.
- [85] Microsoft. Windows Drivers in Rust[EB/OL]. 2025. <https://github.com/microsoft/windows-drivers-rs>.
- [86] Zhang I, Raybuck A, Patel P, et al. The Demikernel Datapath OS Architecture for Microsecond-scale Datacenter Systems[C]. SOSP '21: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. Virtual Event, Germany: Association for Computing Machinery, 2021: 195-211.
- [87] Tu S, Zheng W, Kohler E, et al. Speedy transactions in multicore in-memory databases[C]. SOSP '13: Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles. Farmington, Pennsylvania: Association for Computing Machinery, 2013: 18-32.
- [88] Panda A, Han S, Jang K, et al. NetBricks: Taking the V out of NFV[C]. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). Savannah, GA: USENIX Association, 2016: 203-216.
- [89] Golm M, Felser M, Wawersich C, et al. The JX Operating System[C]. ATEC '02: Proceedings of the General Track of the Annual Conference on USENIX Annual Technical Conference. USA: USENIX Association, 2002: 45-58.
- [90] DWARF. The DWARF Debugging Standard[EB/OL]. 2025. <http://dwarfstd.org/>.
- [91] Hu K, Huang W, Wang L, et al. Unishyper: A Rust-based unikernel enhancing reliability and efficiency of embedded systems[J]. Journal of Systems Architecture, 2024, 153: 103199.
- [92] McKenney P E, Fernandes J, Boyd-Wickizer S, et al. RCU Usage In the Linux Kernel: Eighteen Years Later[J]. SIGOPS Oper. Syst. Rev., 2020, 54(1): 47-63.
- [93] StarFive. <https://github.com/starfive-tech/VisionFive2>. 2025.

- [94] Vahldiek-Oberwagner A, Elnikety E, Duarte N O, et al. ERIM: Secure, Efficient In-process Isolation with Protection Keys (MPK)[C]. 28th USENIX Security Symposium (USENIX Security 19). Santa Clara, CA: USENIX Association, 2019: 1221-1238.
- [95] Hedayati M, Gravani S, Johnson E, et al. Hodor: Intra-Process Isolation for High-Throughput Data Plane Libraries[C]. 2019 USENIX Annual Technical Conference (USENIX ATC 19). Renton, WA: USENIX Association, 2019: 489-504.
- [96] Baumann A, Heiser G, Appavoo J, et al. Providing dynamic update in an operating system[C]. ATEC '05: Proceedings of the Annual Conference on USENIX Annual Technical Conference. Anaheim, CA: USENIX Association, 2005: 32.

攻读学位期间发表论文与研究成果清单